

Service Extension for Tencent

Project info

Project information:

Project name

teg-cdn-test

Project number

363072182815

Project ID

teg-cdn-test

Deployment steps

1.create a artifact registry repository on console

[← Images for service-extension](#) [DELETE](#) [EDIT REPOSITORY](#) [SETUP INSTRUCTIONS](#)

asia-southeast1-docker.pkg.dev > teg-cdn-test > service-extension

Repository Details

Format	Docker
Type	Standard

Repository link :

asia-southeast1-docker.pkg.dev/teg-cdn-test/service-extension

2. Compile source code in dev environment

Cargo.toml

```
[package]
```

```
name = "my-wasm-plugin"
```

```
version = "0.1.0"
```

```
edition = "2023"
```

```
[dependencies]
```

```
proxy-wasm = "0.2"
```

```
log = "0.4"
```

```
md5= "0.7.0"
```

```
url= "2.3.1"
```

```
[lib]
```

```
crate-type = ["cdylib"]
```

```
cargo build --release --target wasm32-wasi
```

```
docker build --no-cache --platform wasm -t tencent-md5-parameter-plugin .
```

```
docker tag tencent-md5-parameter-plugin\
asia-southeast1-docker.pkg.dev/teg-cdn-test/service-extension/tencent-md5-parameter-pl
ugin:v1
```

```
docker push \
asia-southeast1-docker.pkg.dev/teg-cdn-test/service-extension/tencent-md5-parameter-pl
ugin:v1
```

```
/create plugin first
```

```
gcloud alpha service-extensions wasm-plugins create tencent-md5-parameter-plugin
```

```
/create version in plugin
```

```
gcloud alpha service-extensions wasm-plugin-versions create parameter-plugin-v1 \
--wasm-plugin=tencent-md5-parameter-plugin \
--image="asia-southeast1-docker.pkg.dev/teg-cdn-test/service-extension/tencent-md5-par
ameter-plugin:v1"
```

/set main version for plugin

```
gcloud alpha service-extensions wasm-plugins update tencent-md5-parameter-plugin  
--main-version=parameter-plugin-v1
```

/create action

```
gcloud alpha service-extensions wasm-actions create tencent-md5-parameter-action \  
  --wasm-plugin=tencent-md5-parameter-plugin
```

/attach network action to media cdn

```
gcloud alpha edge-cache services export md5-parameter \  
  --destination=md5-parameter.yaml
```

logConfig:

enable: true

sampleRate: 1.0

name: projects/teg-cdn-test/locations/global/edgeCacheServices/md5-parameter

routing:

hostRules:

- hosts:

- '*'

pathMatcher: path-matcher-0

pathMatchers:

- name: path-matcher-0

routeRules:

- headerAction: {}

matchRules:

- prefixMatch: /

origin: projects/363072182815/locations/global/edgeCacheOrigins/md5-test

priority: 10

routeAction:

cdnPolicy:

cacheKeyPolicy: {}

cacheMode: CACHE_ALL_STATIC

defaultTtl: 3600s

maxTtl: 86400s

signedRequestMode: DISABLED

wasmAction: "tencent-md5-parameter-action"

Add something like `wasmAction: "my-wasm-action-resource"` nested in `routeAction`

```
gcloud alpha edge-cache services import md5-parameter \
--source=md5-parameter.yaml
```

```
gcloud alpha service-extensions wasm-plugin-versions list
--wasm-plugin=my-wasm-plugin-resource
```

```
gcloud alpha service-extensions wasm-plugin-versions delete my-version-2
--wasm-plugin=my-wasm-plugin-resource
```

Test result :

curl

["http://34.104.34.169/live/v1/v2/v3/streamid.m3u8?txSecret=8d9c532e0a2c543fca3d02de669ce2b&txTime=1704277794"](http://34.104.34.169/live/v1/v2/v3/streamid.m3u8?txSecret=8d9c532e0a2c543fca3d02de669ce2b&txTime=1704277794)

Run

```
echo -n private_keytest1704277795 |md5sum
```

Got

19c2ae31923c73f92cdda8a8d090912a

curl

["http://34.104.34.169/test.m3u8?txSecret=19c2ae31923c73f92cdda8a8d090912a&txTime=1704277795"](http://34.104.34.169/test.m3u8?txSecret=19c2ae31923c73f92cdda8a8d090912a&txTime=1704277795)

curl

"http://34.104.34.169/test.m3u8?txSecret=19c2ae31923c73f92cdda8a8d090912a&txTime=1704277795" -v

* Trying 34.104.34.169:80...

* Connected to 34.104.34.169 (34.104.34.169) port 80 (#0)

> GET /test.m3u8?txSecret=19c2ae31923c73f92cdda8a8d090912a&txTime=1704277795

HTTP/1.1

> Host: 34.104.34.169

> User-Agent: curl/7.74.0

> Accept: */*

>

* Mark bundle as not supporting multiuse

< HTTP/1.1 200 OK
< server: Google-Edge-Cache
< date: Thu, 18 May 2023 05:21:39 GMT
< content-type: application/vnd.apple.mpegurl
< content-length: 10
< last-modified: Thu, 18 May 2023 05:16:15 GMT
< etag: "6465b49f-a"
< accept-ranges: bytes
< x-request-id: 1ce1046e-9138-48bf-997e-fb1e376f7a94
< x-xss-protection: 0
< x-frame-options: SAMEORIGIN
< age: 0
< x-content-type-options: nosniff
< cache-control: public,max-age=1200
<
test.m3u8

Change to Hex timestamp and only apply to m3u8

```
logConfig:
  enable: true
  sampleRate: 1.0
name: projects/teg-cdn-test/locations/global/edgeCacheServices/md5-parameter
routing:
  hostRules:
    - hosts:
        - '*'
      pathMatcher: path-matcher-0
  pathMatchers:
    - name: path-matcher-0
      routeRules:
        - headerAction: {}
          matchRules:
            - ignoreCase: true
              pathTemplateMatch: /**m3u8
          origin: projects/363072182815/locations/global/edgeCacheOrigins/md5-test
          priority: 10
          routeAction:
            cdnPolicy:
              cacheKeyPolicy: {}
              cacheMode: CACHE_ALL_STATIC
              defaultTtl: 3600s
              maxTtl: 86400s
```

```
signedRequestMode: DISABLED
wasmAction: "tencent-md5-parameter-action"
urlRewrite:
  hostRewrite: 68789.liveplay.myqcloud.com
- headerAction: {}
matchRules:
- prefixMatch: /
origin: projects/363072182815/locations/global/edgeCacheOrigins/md5-test
priority: 11
routeAction:
  cdnPolicy:
    cacheKeyPolicy: {}
    cacheMode: CACHE_ALL_STATIC
    defaultTtl: 3600s
    maxTtl: 86400s
    signedRequestMode: DISABLED
  urlRewrite:
    hostRewrite: 68789.liveplay.myqcloud.com
```

```
use log::info;
use proxy_wasm::traits::*;
use proxy_wasm::types::*;
use std::time::{SystemTime, UNIX_EPOCH};
use md5;
use url::Url;
use std::collections::HashMap;
proxy_wasm::main! {{
  proxy_wasm::set_log_level(LogLevel::Trace);
  proxy_wasm::set_http_context(|_,_| -> Box<dyn HttpContext> { Box::new(DemoPlugin) });
}}
pub fn parse_url_fullpath(url_string:&str,private_key:&str) -> Result<String,String>
{
  //split url to vec
  let vec: Vec<&str> = url_string.split("/").collect::<Vec<&str>>();
```

```

//get fileid from url
let filename=vec[vec.len()-1].split(".").collect::<Vec<&str>>();
let fileid=filename[0];
info!("{}",format!("file is {}",fileid));
let pseudo_url="http://localhost".to_string()+url_string;
info!("{}",format!("pseudo_url is {}",pseudo_url));
let parsed_url = Url::parse(&pseudo_url).unwrap();
let hash_query: HashMap<_, _> = parsed_url.query_pairs().into_owned().collect();

if !hash_query.get("txSecret").is_some() {
    return Err(format!("txSecret is missing"));
}
if !hash_query.get("txTime").is_some() {
    return Err(format!("txTime is missing"));
}
info!("{}",format!("txSecret is {}",hash_query.get("txSecret").unwrap()));
info!("{}",format!("txTime is {}",hash_query.get("txTime").unwrap()));
let token=hash_query.get("txSecret").unwrap();

//get current timestamp
let now = SystemTime::now();
let since_the_epoch: u64 = now
    .duration_since(UNIX_EPOCH)
    .expect("Time went backwards").as_secs();

//get timestamp from url
let timestamp;
let timestamp_s=hash_query.get("txTime").unwrap();
//let f=hash_query.get("txTime").unwrap().parse::<u64>() ;
let f=u64::from_str_radix(timestamp_s,16);
match f {

```

```

    Ok(_v) => {timestamp=f.unwrap()}
    Err(_f)=> {
        return Err(format!("timestamp is invalid:{}",timestamp_s));
    }
}

info!("{}",format!("get timestamp is {}",timestamp_s));
//check timestamp,if current is greater than timestamp,will panic

if since_the_epoch > timestamp {
    return Err(format!("timestamp is expired:{},{},{}",timestamp_s,timestamp,since_the_epoch))
}

//get real backend url and sign url from path
let mut sign_url=private_key.to_string();
sign_url+=&fileid.to_string();
sign_url+=&timestamp_s;
info!("{}",format!("sign value is {}",sign_url));
let digest =format!("{}",md5::compute(sign_url));
if token.eq(&digest) {
    Ok(url_string.to_string())
} else {

    return Err(format!("token is invalid:{},correct should be:{}",url_string,digest))

}
}

struct DemoPlugin;

impl HttpContext for DemoPlugin {
    fn on_http_request_headers(&mut self, _: usize, _: bool) -> Action {
        let http_path=self.get_http_request_header(":path").unwrap();

```



```

let result=parse_url_fullpath(&http_path,&"private_key");
match result {
    Ok(v) => {
        self.set_http_request_header(":path",Some(&v));
        info!("Access granted.");
    }
    Err(v) => {
        info!("{}",format!("return error:{}",v));
        self.send_http_response(403, vec![], Some(b"Access forbidden.\n"));
        return Action::Pause;
    }
}

Action::Continue
}
}

impl Context for DemoPlugin {}

```

Test result :

curl

"http://34.104.34.169/live/v1/v2/v3/streamid.m3u8?txSecret=afcb4ae5ea1b008e080cd21f3b77a4a9&txTime=64FE6D2B" -v

* Trying 34.104.34.169:80...

* Connected to 34.104.34.169 (34.104.34.169) port 80 (#0)

> GET

/live/v1/v2/v3/streamid.m3u8?txSecret=afcb4ae5ea1b008e080cd21f3b77a4a9&txTime=64FE6D2B HTTP/1.1

> Host: 34.104.34.169

> User-Agent: curl/7.74.0

> Accept: */*

>

* Mark bundle as not supporting multiuse

< HTTP/1.1 200 OK

< server: Google-Edge-Cache

```
< content-length: 5
< accept-ranges: bytes
< x-request-id: 918c2043-16a6-4194-ba90-8a4555a2ec0e
< x-xss-protection: 0
< x-frame-options: SAMEORIGIN
< x-content-type-options: nosniff
< date: Thu, 18 May 2023 09:08:53 GMT
< age: 95
< last-modified: Thu, 18 May 2023 03:47:21 GMT
< etag: "64659fc9-5"
< content-type: application/vnd.apple.mpegurl
< cache-control: public,max-age=1200
<
test
* Connection #0 to host 34.104.34.169 left intact
```

```
curl "http://34.104.34.169/live/v1/v2/v3/segment1.ts?aaa"
```

```
curl "http://34.104.34.169/live/v1/v2/v3/segment1.ts?aaa" -v
* Trying 34.104.34.169:80...
* Connected to 34.104.34.169 (34.104.34.169) port 80 (#0)
> GET /live/v1/v2/v3/segment1.ts?aaa HTTP/1.1
> Host: 34.104.34.169
> User-Agent: curl/7.74.0
> Accept: */*
>
* Mark bundle as not supporting multiuse
< HTTP/1.1 200 OK
< server: Google-Edge-Cache
< content-length: 10
< access-control-allow-origin: www.test.com
< access-control-allow-methods: GET, POST, OPTIONS
< access-control-allow-headers:
DNT,User-Agent,X-Requested-With,If-Modified-Since,Cache-Control,Content-Type,Range
< access-control-expose-headers: Content-Length,Content-Range
< accept-ranges: bytes
< x-request-id: a3a529a6-2462-4db1-92d5-752d7e3b481d
< x-xss-protection: 0
< x-frame-options: SAMEORIGIN
< date: Thu, 18 May 2023 09:12:47 GMT
< last-modified: Thu, 18 May 2023 09:11:40 GMT
```

```
< etag: "6465ebcc-a"
< content-type: video/mp2t
< age: 11
< cache-control: public,max-age=3600
<
segment 1
* Connection #0 to host 34.104.34.169 left intact
```

MD5 Version 1

Schema:

```
/md5(key+path('/v1/v2../vn')+txTime)/txTime/v1/v2../vn/streamid.m3u8
```

```
use log::info;
use proxy_wasm::traits::*;
use proxy_wasm::types::*;
use std::time::{SystemTime, UNIX_EPOCH};
use md5;

proxy_wasm::main! {{
    proxy_wasm::set_log_level(LogLevel::Trace);
    proxy_wasm::set_http_context(|_,_| -> Box<dyn HttpContext> { Box::new(DemoPlugin) });
}}

pub fn parse_url_fullpath(url_string:&str,private_key:&str) -> Result<String,String>
{
    //split url to vec
    let vec: Vec<&str> = url_string.split("/").collect::<Vec<&str>>();
    //get token from url
    let token=vec[1].to_string();
    //get current timestamp
```

```

let now = SystemTime::now();
let since_the_epoch = now
    .duration_since(UNIX_EPOCH)
    .expect("Time went backwards").as_secs();

//get timestamp from url
let timestamp;
let f=vec[2].parse::<u64>() ;
match f {
    Ok(_v) => {timestamp=f.unwrap()}
    Err(_f)=> {
        return Err(format!("timestamp is invalid:{}",vec[2]));
    }
}

info!("{}",format!("get timestamp is {}",timestamp));
//check timestamp,if current is greater than timestamp,will panic
if since_the_epoch > timestamp {
    return Err(format!("timestamp is invalid:{}",vec[2]))
}

//get real backend url and sign url from path

let mut backend_url="".to_string();
let mut sign_url=private_key.to_string();
for (index,el) in vec[0..].iter().enumerate() {

    if index >2 {
        backend_url+=&"/".to_string();
        backend_url+=&el;

        if index>2 && index<vec.len()-1 {
            sign_url+=&"/".to_string();

```

```

        sign_url += &el;

    }
}

}

sign_url += &timestamp.to_string();
info!("{}", format!("backend url is {}", backend_url));
info!("{}", format!("sign value is {}", sign_url));

let digest = format!("{:x}", md5::compute(sign_url));
if token.eq(&digest) {
    Ok(backend_url.to_string())
} else {
    return Err(format!("token is invalid: {}, correct should be: {}", url_string, digest))
}
}
}

```

```

struct DemoPlugin;

impl HttpContext for DemoPlugin {
    fn on_http_request_headers(&mut self, _: usize, _: bool) -> Action {
        let http_path = self.get_http_request_header(":path").unwrap();
        let result = parse_url_fullpath(&http_path, &"private_key");
        match result {
            Ok(v) => {
                self.set_http_request_header(":path", Some(&v));
                self.set_http_request_header("host", Some("10.170.0.10"));
                info!("Access granted.");
            }
        }
    }
}

```

```

    }
    Err(v) => {
        info!("{}",format!("return error:{}",v));
        self.send_http_response(403, vec![], Some(b"Access forbidden.\n"));
        return Action::Pause;
    }
}
Action::Continue
}

}

impl Context for DemoPlugin {}

```

MD5 Version 2

Schema:

/live/streamid.m3u8?txSecret=md5(key+streamid+txTime)&txTime=16

```

use log::info;
use proxy_wasm::traits::*;
use proxy_wasm::types::*;
use std::time::{SystemTime, UNIX_EPOCH};
use md5;
use url::Url;

```

```

use std::collections::HashMap;
proxy_wasm::main! {{
proxy_wasm::set_log_level(LogLevel::Trace);
proxy_wasm::set_http_context(|_,_| -> Box<dyn HttpContext> { Box::new(DemoPlugin) });
}}

pub fn parse_url_fullpath(url_string:&str,private_key:&str) -> Result<String,String>
{
    //split url to vec
    let vec: Vec<&str> = url_string.split("/").collect::<Vec<&str>>();
    //get fileid from url
    let filename=vec[vec.len()-1].split(".").collect::<Vec<&str>>();
    let fileid=filename[0];
    info!("{}",format!("file is {}",fileid));
    let presudo_url="http://localhost".to_string()+url_string;
    info!("{}",format!("presudo_url is {}",presudo_url));
    let parsed_url = Url::parse(&presudo_url).unwrap();
    let hash_query: HashMap<_,_> = parsed_url.query_pairs().into_owned().collect();

    if !hash_query.get("txSecret").is_some() {
        return Err(format!("txSecret is missing"));
    }
    if !hash_query.get("txTime").is_some() {
        return Err(format!("txTime is missing"));
    }
    info!("{}",format!("txSecret is {}",hash_query.get("txSecret").unwrap()));
    info!("{}",format!("txTime is {}",hash_query.get("txTime").unwrap()));
    let token=hash_query.get("txSecret").unwrap();
    //get current timestamp
    let now = SystemTime::now();
    let since_the_epoch = now

```

```

.duration_since(UNIX_EPOCH)
.expect("Time went backwards").as_secs();
//get timestamp from url
let timestamp;
let timestamp_s=hash_query.get("txTime").unwrap();
let f=hash_query.get("txTime").unwrap().parse::<u64>() ;
match f {
    Ok(_v) => {timestamp=f.unwrap()}
    Err(_f)=> {
        return Err(format!("timestamp is invalid:{}",timestamp_s));
    }
}
info!("{}",format!("get timestamp is {}",timestamp_s));
//check timestamp,if current is greater than timestamp,will panic
if since_the_epoch > timestamp {
    return Err(format!("timestamp is invalid:{}",timestamp_s))
}
//get real backend url and sign url from path
let mut sign_url=private_key.to_string();
sign_url+=&fileid.to_string();
sign_url+=&timestamp.to_string();
info!("{}",format!("sign value is {}",sign_url));
let digest =format!("{}",md5::compute(sign_url));
if token.eq(&digest) {
    Ok(url_string.to_string())
} else {
    return Err(format!("token is invalid:{},correct should be:{}",url_string,digest))
}

```



```

}

struct DemoPlugin;

impl HttpContext for DemoPlugin {

fn on_http_request_headers(&mut self, _: usize, _: bool) -> Action {
    let http_path=self.get_http_request_header(":path").unwrap();
    let result=parse_url_fullpath(&http_path,&"private_key");
    match result {
        Ok(v) => {
            self.set_http_request_header(":path",Some(&v));
            self.set_http_request_header("host",Some("10.170.0.10"));
            info!("Access granted.");
        }
        Err(v) => {
            info!("{}",format!("return error:{}",v));
            self.send_http_response(403, vec![], Some(b"Access forbidden.\n"));
            return Action::Pause;
        }
    }
}

Action::Continue
}
}

impl Context for DemoPlugin {}

```

MD5 Version 3

Schema:

/md5(key+streamid+txTime)/txTime/live/streamid.m3u8

```
use log::info;
use proxy_wasm::traits::*;
use proxy_wasm::types::*;
use std::time::{SystemTime, UNIX_EPOCH};
use md5;

proxy_wasm::main! {{
    proxy_wasm::set_log_level(LogLevel::Trace);
    proxy_wasm::set_http_context(|_,_|->Box<dyn HttpContext> { Box::new(DemoPlugin) });
}}

pub fn parse_url_fullpath(url_string:&str,private_key:&str) -> Result<String,String>
{
    //split url to vec
    let vec: Vec<&str> = url_string.split("/").collect::<Vec<&str>>();

    //get token from url
    let token=vec[1].to_string();
    info!("{}",format!("token is {}",token));

    //get fileid from url
    let filename=vec[vec.len()-1].split(".").collect::<Vec<&str>>();
    let fileid=filename[0];
    info!("{}",format!("file is {}",fileid));

    //get current timestamp
    let now = SystemTime::now();
    let since_the_epoch = now
        .duration_since(UNIX_EPOCH)
```

```

.expect("Time went backwards").as_secs();
//get timestamp from url
let timestamp;
let f=vec[2].parse::<u64>();
match f {
    Ok(_v) => {timestamp=f.unwrap()}
    Err(_f)=> {
        return Err(format!("timestamp is invalid:{}",vec[2]));
    }
}

info!("{}",format!("get timestamp is {}",timestamp));
//check timestamp,if current is greater than timestamp,will panic
if since_the_epoch > timestamp {
    return Err(format!("timestamp is invalid:{}",vec[2]))
}

//get real backend url and sign url from path
let mut backend_url="".to_string();
let sign_url=private_key.to_string()+fileid+&timestamp.to_string();
for (index,el) in vec[0..].iter().enumerate() {
    if index > 2 {
        backend_url+=&"/".to_string();
        backend_url+=&el;
    }
}

info!("{}",format!("backend url is {}",backend_url));
info!("{}",format!("sign value is {}",sign_url));
let digest =format!("{:x}",md5::compute(sign_url));
if token.eq(&digest) {

```

```

    Ok(backend_url.to_string())
} else {
    return Err(format!("token is invalid:{}",url_string,digest))
}
}
}

struct DemoPlugin;

impl HttpContext for DemoPlugin {
fn on_http_request_headers(&mut self, _: usize, _: bool) -> Action {
    let http_path=self.get_http_request_header(":path").unwrap();
    let result=parse_url_fullpath(&http_path,&"private_key");
    match result {
    Ok(v) => {
        self.set_http_request_header(":path",Some(&v));
        self.set_http_request_header("host",Some("10.170.0.10"));
        info!("Access granted.");
    }
    Err(v) => {
        info!("{}",format!("return error:{}",v));
        self.send_http_response(403, vec![], Some(b"Access forbidden.\n"));
        return Action::Pause;
    }
}
}

Action::Continue
}
}

impl Context for DemoPlugin {}

```

Deploy note

```
gcloud alpha service-extensions wasm-plugins create tencent-wasm-plugin-md5-path
gcloud alpha service-extensions wasm-plugins create
tencent-wasm-plugin-md5-paratemer
gcloud alpha service-extensions wasm-plugins create tencent-wasm-plugin-md5-file
```

```
gcloud alpha service-extensions wasm-actions create tencent-md5-path \
  --wasm-plugin=tencent-wasm-plugin-md5-path
```

```
gcloud alpha service-extensions wasm-actions create tencent-md5-paratemer \
  --wasm-plugin=tencent-wasm-plugin-md5-paratemer
```

```
gcloud alpha service-extensions wasm-actions create tencent-md5-file \
  --wasm-plugin=tencent-wasm-plugin-md5-file
```

```
cargo build --release --target wasm32-wasi
docker build --no-cache --platform wasm -t tencent-wasm-plugin-md5-path .
docker tag tencent-wasm-plugin-md5-path \
asia-southeast1-docker.pkg.dev/jian-lin-aroglis-common/my-docker-repo/tencent-wasm-pl
ugin-md5-path:v1
docker push \
asia-southeast1-docker.pkg.dev/jian-lin-aroglis-common/my-docker-repo/tencent-wasm-pl
ugin-md5-path:v1
```

```
gcloud alpha service-extensions wasm-plugin-versions create
tencent-wasm-plugin-md5-path-version-1 \
  --wasm-plugin=tencent-wasm-plugin-md5-path \
```

```
--image="asia-southeast1-docker.pkg.dev/jian-lin-aroglis-common/my-docker-repo/tencent
-wasm-plugin-md5-path:v1"
```

```
gcloud alpha service-extensions wasm-plugins update tencent-wasm-plugin-md5-path \
  --main-version=tencent-wasm-plugin-md5-path-version-1
```

```
cargo build --release --target wasm32-wasi
docker build --no-cache --platform wasm -t tencent-wasm-plugin-md5-paratemer .
docker tag tencent-wasm-plugin-md5-paratemer \
asia-southeast1-docker.pkg.dev/jian-lin-aroglis-common/my-docker-repo/tencent-wasm-pl
ugin-md5-paratemer:v1
docker push \
asia-southeast1-docker.pkg.dev/jian-lin-aroglis-common/my-docker-repo/tencent-wasm-pl
```

ugin-md5-paratemer:v1

```
gcloud alpha service-extensions wasm-plugin-versions create
tencent-wasm-plugin-md5-paratemer-version-1 \
  --wasm-plugin=tencent-wasm-plugin-md5-paratemer \
```

```
--image="asia-southeast1-docker.pkg.dev/jian-lin-aroglis-common/my-docker-repo/tencent
-wasm-plugin-md5-paratemer:v1"
```

```
gcloud alpha service-extensions wasm-plugins update
tencent-wasm-plugin-md5-paratemer \
  --main-version=tencent-wasm-plugin-md5-paratemer-version-1
```

```
cargo build --release --target wasm32-wasi
docker build --no-cache --platform wasm -t tencent-wasm-plugin-md5-file .
docker tag tencent-wasm-plugin-md5-file \
asia-southeast1-docker.pkg.dev/jian-lin-aroglis-common/my-docker-repo/tencent-wasm-pl
ugin-md5-file:v1
docker push \
asia-southeast1-docker.pkg.dev/jian-lin-aroglis-common/my-docker-repo/tencent-wasm-pl
ugin-md5-file:v1
```

```
gcloud alpha service-extensions wasm-plugin-versions create
tencent-wasm-plugin-md5-file-version-1 \
  --wasm-plugin=tencent-wasm-plugin-md5-file \
```

```
--image="asia-southeast1-docker.pkg.dev/jian-lin-aroglis-common/my-docker-repo/tencent
-wasm-plugin-md5-file:v1"
```

```
gcloud alpha service-extensions wasm-plugins update tencent-wasm-plugin-md5-file \
  --main-version=tencent-wasm-plugin-md5-file-version-1
```

check:

```
gcloud alpha service-extensions wasm-plugin-versions list
--wasm-plugin=tencent-wasm-plugin-md5-path
gcloud alpha service-extensions wasm-plugin-versions list
--wasm-plugin=tencent-wasm-plugin-md5-paratemer
gcloud alpha service-extensions wasm-plugin-versions list
--wasm-plugin=tencent-wasm-plugin-md5-file
```

```
gcloud edge-cache services export tencent-md5-path \
  --destination=tencent-md5-path.yaml
```

wasmAction: "tencent-md5-path"

```

gcloud alpha edge-cache services import tencent-md5-path \
--source=tencent-md5-path.yaml

gcloud alpha edge-cache services export tencent-md5-parameter \
--destination=tencent-md5-parameter.yaml

add wasmAction: "tencent-md5-paratemer"

gcloud edge-cache services import tencent-md5-parameter \
--destination=tencent-md5-parameter.yaml

gcloud alpha edge-cache services import tencent-md5-parameter \
--source=tencent-md5-parameter.yaml

gcloud edge-cache services export tencent-md5-file \
--destination=tencent-md5-file.yaml

add wasmAction: "tencent-md5-file"

gcloud alpha edge-cache services import tencent-md5-file \
--source=tencent-md5-file.yaml

```

Test Result :

curl

<http://34.104.35.121/3e812dce64441cfac511064fad4fbf33/1704277794/v1/v2/v3/streamid.m3u8>

```

[jianln-macbookpro1:~ jianln$ curl http://34.104.35.121/token/1704277794/v1/v2/v3/streamid.m3u8
Access forbidden.
[jianln-macbookpro1:~ jianln$ curl http://34.104.35.121/3e812dce64441cfac511064fad4fbf33/1704277794/v1/v2/v3/s
treamid.m3u8
::ffff:172.253.237.3,/v1/v2/v3/streamid.m3u8,GET,{"user-agent":"curl/7.79.1","accept":"*/*","x-forwarded-prot
o":"http","x-request-id":"68baea67-cbe2-4991-8d21-c240e2c83ad4","x-media-edge-logging-info":"CAESA0hLRxoBLw",
"cdn-loop":"google","host":"34.104.35.121","connection":"keep-alive"},jianln-macbookpro1:~ jianln$ █

```

curl <http://34.104.37.161/8d9c532e0a2c543fca3d02de669cec2b/1704277794/live/streamid.m3u8>

```
jianln-macbookpro1:~ jianln$ curl http://34.104.37.161/token/1704277794/live/streamid.m3u8
Access forbidden.
jianln-macbookpro1:~ jianln$ curl http://34.104.37.161/8d9c532e0a2c543fca3d02de669cec2b/1704277794/live/streamid.m3u8
::ffff:172.253.4.2,/live/streamid.m3u8,GET,{\"user-agent\":\"curl/7.79.1\",\"accept\":\"/*/*\", \"x-forwarded-proto\":\"http\", \"x-request-id\":\"3a646a88-d810-47a5-a10b-29d609710583\", \"x-media-edge-logging-info\":\"CAESA0hLRxoBLw\", \"cdn-loop\":\"google\", \"host\":\"34.104.37.161\", \"connection\":\"keep-alive\"},jianln-macbookpro1:~ jianln$
```

curl

["http://34.104.35.237/live/v1/v2/v3/streamid.m3u8?txSecret=8d9c532e0a2c543fca3d02de669cec2b&txTime=1704277794"](http://34.104.35.237/live/v1/v2/v3/streamid.m3u8?txSecret=8d9c532e0a2c543fca3d02de669cec2b&txTime=1704277794)

```
jianln-macbookpro1:~ jianln$ curl "http://34.104.35.237/live/v1/v2/v3/streamid.m3u8?txSecret=8d9c532e0a2c543fca3d02de669cec2b&txTime=1704277794"
::ffff:172.253.237.2,/live/v1/v2/v3/streamid.m3u8?txSecret=8d9c532e0a2c543fca3d02de669cec2b&txTime=1704277794,GET,{\"user-agent\":\"curl/7.79.1\",\"accept\":\"/*/*\", \"x-forwarded-proto\":\"http\", \"x-request-id\":\"3bf184d0-915c-4851-8d2e-e7373d6eae9f\", \"x-media-edge-logging-info\":\"CAESA0hLRxoBLw\", \"cdn-loop\":\"google\", \"host\":\"34.104.35.237\", \"
```

MD5 Version 4

Schema:

/txSecret/txTime/path/fileName.m3u8

Customer requirement

鉴权需求：

只有m3u8需要鉴权。ts不鉴权。

url格式：

/txSecret/txTime/path/fileName.m3u8

鉴权规则：

两个鉴权字段 txSecret，txTime 放到了url里面，如：

/txSecret/txTime/path/fileName.m3u8，

鉴权字段说明：

txTime：和官网一致，16进制秒级，大小写都要支持，有效范围是：txTime + 86400s。

txSecret：md5(key+streamId+txTime)，其中txTime不要做大小写转换，用url里面原始的。

streamId：先尝试将fileName作为streamId，如果鉴权失败，再尝试判断fileName是否带下划线，如果有，去掉fileName中最后一个下划线及其后面的字符再做一次判断。如果都检验失败，则失败。

如：/live/6611082f-f0ac-4b90-835a-bd5c4966e047_360pprod.m3u8？ 这种带了一个下划线 '_'，streamId为 6611082f-f0ac-4b90-835a-bd5c4966e047

/live/test_1080x960_360pprod.m3u8? 这种带了两个下划线 '_' 要
剔除掉, streamId为 test_1080x960
 /live/6611082f-f0ac-4b90-8352342342.m3u8? 这种不带, streamId为
6611082f-f0ac-4b90-8352342342
 key: 有主备两个key

这样的话鉴权的时候可能需要尝试多次

回源url改写:

回源的时候需要进行url重写去掉 txSecret, txTime 两个字段, 改写成
/path/fileName.m3u8 —— 已经验证gcp是支持的

缓存key需求:

要去掉uri中的前两跳(即 /txSecret/txTime), 防止鉴权参数影响命中率。

/md5(key+streamid+txTime)/txTime/live/streamid.m3u8

Code

```
use log::info;
use proxy_wasm::traits::*;
use proxy_wasm::types::*;
use std::time::{SystemTime, UNIX_EPOCH};
use md5;
use lazy_static::lazy_static;
use regex::Regex;

proxy_wasm::main! {{
proxy_wasm::set_log_level(LogLevel::Trace);
proxy_wasm::set_root_context(|_| -> Box<dyn RootContext> {
Box::new(DemoPluginRoot {
secret: String::new(),
})
});
```

```

}}

struct DemoPluginRoot {
secret: String,
}

impl RootContext for DemoPluginRoot {
fn on_configure(&mut self, _: usize) -> bool {
if let Some(config_bytes) = self.get_plugin_configuration() {
self.secret = String::from_utf8(config_bytes).unwrap()
}
true
}

fn create_http_context(&self, _: u32) -> Option<Box<dyn HttpContext>> {
Some(Box::new(DemoPlugin {
secret: self.secret.clone(),
}))
}

fn get_type(&self) -> Option<ContextType> {
Some(ContextType::HttpContext)
}
}

impl Context for DemoPluginRoot {}

fn extract_streamid(input: &str) -> Option<&str> {
lazy_static! {
static ref RE: Regex = Regex::new(r"(?x)

```

```

^(?P<streamid>[^\s]+)_
.*$
").unwrap();
}
RE.captures(input).and_then(|cap| {
cap.name("streamid").map(|streamid| streamid.as_str())
})
}

pub fn parse_url_fullpath(url_string:&str,private_key:&str) -> Result<String,String>
{
//split key array to keys
let vec_key: Vec<&str> = private_key.split("|").collect::<Vec<&str>>();
if vec_key.len()<2{
info!("{}",format!("key format is invalid,key len is {}",vec_key.len()));
}

let key_primary=vec_key[0];
let key_secondary=vec_key[1];

//split url to vec
let vec: Vec<&str> = url_string.split("/").collect::<Vec<&str>>();
if vec.len()<2{
info!("{}",format!("request format is invalid,path segment len is {}",vec.len()));
}

//get token from url
let token=vec[1].to_string();
info!("{}",format!("token is {}",token));

//get fileid from url
let filename=vec[vec.len()-1].split(".").collect::<Vec<&str>>();
let fileid=filename[0];
info!("{}",format!("file is {}",fileid));
}

```

```

//get current timestamp
let now = SystemTime::now();
let current = now
.duration_since(UNIX_EPOCH)
.expect("Time went backwards").as_secs();

//get timestamp from url
let timestamp;
let f=u64::from_str_radix(vec[2],16);
match f {
Ok(_v) => {timestamp=f.unwrap()}
Err(_f)=> {
return Err(format!("timestamp is invalid in parse from hex to u64:{}",vec[2]));
}
}

info!("{}",format!("get timestamp is {}",vec[2]));

//check timestamp,if current is greater than timestamp then exist
//if since_the_epoch < timestamp {
// return Err(format!("timestamp exceed current timestamp
: {}, {}, {}",vec[2],since_the_epoch,timestamp))
// }

if current>(timestamp+86400) {
return Err(format!("timestamp is expired : {}, {}, {}",vec[2],current,timestamp))
}

//get real backend url and sign url from path
let mut backend_url="".to_string();
for (index,el) in vec[0..].iter().enumerate() {
if index >2 {

```

```

backend_url+=&"/".to_string();
backend_url+=&el;
}
}

info!("{}",format!("backend url is {}",backend_url));


// first round check

let mut sign_url=key_primary.to_string()+fileid+&vec[2].to_string();
info!("{}",format!("first sign url:{}",sign_url));
let mut digest=format!("{:x}",md5::compute(sign_url));
info!("{}",format!("first token:{}",digest));
if token.eq(&digest) {
return Ok(backend_url.to_string())
} else {
info!("{}",format!("first token check failed"));
let streamid=extract_streamid(fileid);
// second round check
if streamid.is_some() {

sign_url=key_primary.to_string()+&streamid.unwrap().to_string()+&vec[2].to_string();
info!("{}",format!("second sign url:{}",sign_url));
digest=format!("{:x}",md5::compute(sign_url));
info!("{}",format!("second token url:{}",digest));


if token.eq(&digest) {
return Ok(backend_url.to_string())
}else {
info!("{}",format!("all token for primary key check is invalid"));

```

```

}

} else {

info!("{}",format!("failed to extract streamid:{}",fileid));

}

}

// third round check
sign_url=key_secondary.to_string()+fileid+&vec[2].to_string();
info!("{}",format!("third sign url:{}",sign_url));
digest =format!("{}",md5::compute(sign_url));
info!("{}",format!("third token :{}",digest));

if token.eq(&digest) {
return Ok(backend_url.to_string())
} else {
info!("{}",format!(" third round check failed"));
let streamid=extract_streamid(fileid);
if streamid.is_some() {

sign_url=key_secondary.to_string()+&streamid.unwrap().to_string()+&vec[2].to_string();
info!("{}",format!("fourth sign url:{}",sign_url));
digest =format!("{}",md5::compute(sign_url));
info!("{}",format!("fourth token :{}",digest));

if token.eq(&digest) {
return Ok(backend_url.to_string())
}else {
info!("{}",format!("all token for secondary key check is invalid"));
}
}else {

```

```

info!("{}",format!("failed to extract streamid:{}",fileid));
}
}

return Err(format!("all token check invalid"))

}

struct DemoPlugin {
secret: String,
}

impl HttpContext for DemoPlugin {
fn on_http_request_headers(&mut self, _: usize, _: bool) -> Action {
let http_path=self.get_http_request_header(":path").unwrap();
let result=parse_url_fullpath(&http_path,&self.secret);
match result {
Ok(v) => {
self.set_http_request_header(":path",Some(&v));
info!("Access granted.");
}
Err(v) => {
info!("{}",format!("return error:{}",v));
self.send_http_response(403, vec![], Some(b"Access forbidden.\n"));
return Action::Pause;
}
}

Action::Continue
}
}

impl Context for DemoPlugin {}

```



Cargo.toml

```
[package]
name = "my-wasm-plugin"
version = "0.1.0"
edition = "2021"
[dependencies]
proxy-wasm = "0.2"
log = "0.4"
md5 = "0.7.0"
url = "2.3.1"
lazy_static = "1.4.0"
regex = "1.9.0"

[[lib]]
crate-type = ["cdylib"]
```

Deploy note

```
gcloud alpha service-extensions wasm-plugins create tencent-wasm-plugin-md5-v4
```



```
cargo build --release --target wasm32-wasi
docker build --no-cache --platform wasm -t tencent-wasm-plugin-md5-v4 .
docker tag tencent-wasm-plugin-md5-v4 \
asia-southeast1-docker.pkg.dev/jian-lin-aroglis-common/my-docker-repo/tencent-wasm-pl
ugin-md5-v4:v4
docker push \
asia-southeast1-docker.pkg.dev/jian-lin-aroglis-common/my-docker-repo/tencent-wasm-pl
ugin-md5-v4:v4
```

```
gcloud alpha service-extensions wasm-plugin-versions create
tencent-wasm-plugin-md5-v4-20230925 \
  --wasm-plugin=my-wasm-plugin-resource \

--image="asia-southeast1-docker.pkg.dev/jian-lin-aroglis-common/my-docker-repo/tencent
-wasm-plugin-md5-v4:latest" \
  --plugin-config="mytestkey1|mytestkey2"
```

```
gcloud alpha service-extensions wasm-plugins update tencent-wasm-plugin-md5-v4 \
  --main-version=tencent-wasm-plugin-md5-v4-version-1
```

```
gcloud alpha service-extensions wasm-actions create tencent-md5-v4 \
  --wasm-plugin=tencent-wasm-plugin-md5-v4
```

```
gcloud edge-cache services export tencent-md5-v4 \
  --destination=tencent-md5-v4.yaml
```

Add:

```
wasmAction: "tencent-md5-v4"
```

In file "tencent-md5-v4.yaml"

```
gcloud alpha edge-cache services import tencent-md5-v4 \
  --source=tencent-md5-v4.yaml
```

Final yaml file

```
logConfig:
```

```
enable: true
sampleRate: 1.0
name:
projects/jian-lin-aroglis-common/locations/global/edgeCacheServices/tencent-md5-v4
routing:
  hostRules:
    - hosts:
        - www.md5tencent.com
      pathMatcher: path-matcher-0
    - hosts:
        - www.md5mytest.com
      pathMatcher: path-matcher-1
  pathMatchers:
    - name: path-matcher-0
      routeRules:
        - headerAction: {}
      matchRules:
        - ignoreCase: true
          pathTemplateMatch: /**.m3u8
      origin: projects/251868301847/locations/global/edgeCacheOrigins/tencent-md5-v4
      priority: 10
      routeAction:
        cdnPolicy:
          cacheKeyPolicy: {}
          cacheMode: CACHE_ALL_STATIC
          defaultTtl: 3600s
          maxTtl: 86400s
          signedRequestMode: DISABLED
          wasmAction: "tencent-md5-v4"
        urlRewrite:
          hostRewrite: play-live-mojapp.tliveapp.com
    - headerAction: {}
      matchRules:
        - prefixMatch: /
      origin: projects/251868301847/locations/global/edgeCacheOrigins/tencent-md5-v4
      priority: 11
      routeAction:
        cdnPolicy:
          cacheKeyPolicy: {}
          cacheMode: FORCE_CACHE_ALL
          defaultTtl: 3600s
          signedRequestMode: DISABLED
        urlRewrite:
          hostRewrite: play-live-mojapp.tliveapp.com
    - name: path-matcher-1
      routeRules:
```

```
- headerAction: {}
matchRules:
- ignoreCase: true
  pathTemplateMatch: /**.m3u8
  origin: projects/251868301847/locations/global/edgeCacheOrigins/my-orginal-1
  priority: 10
  routeAction:
    cdnPolicy:
      cacheKeyPolicy: {}
      cacheMode: CACHE_ALL_STATIC
      defaultTtl: 3600s
      maxTtl: 86400s
      signedRequestMode: DISABLED
      wasmAction: "tencent-md5-v4"
- headerAction: {}
matchRules:
- prefixMatch: /
  origin: projects/251868301847/locations/global/edgeCacheOrigins/my-orginal-1
  priority: 11
  routeAction:
    cdnPolicy:
      cacheKeyPolicy: {}
      cacheMode: FORCE_CACHE_ALL
      defaultTtl: 3600s
      signedRequestMode: DISABLED
```

Test result :

curl

http://34.104.33.145/b423e0b1e544fe0e54b329c29c67734d/64A654C4/sample_480_640.m3u8

-H "host: www.md5mytest.com"

this is server

MD5 Version 5 (with plugin parameter)

Customer requirement

带参数的version 2, 时间戳使用hex格式

Code

```
use log::info;
use proxy_wasm::traits::*;
use proxy_wasm::types::*;
use std::time::{SystemTime, UNIX_EPOCH};
use md5;
use url::Url;
use std::collections::HashMap;

proxy_wasm::main! {{
    proxy_wasm::set_log_level(LogLevel::Trace);
    proxy_wasm::set_root_context(|_| -> Box<dyn RootContext> {
        Box::new(DemoPluginRoot {
            secret: String::new(),
        })
    });
}}

struct DemoPluginRoot {
    secret: String,
}

impl RootContext for DemoPluginRoot {
    fn on_configure(&mut self, _: usize) -> bool {
        if let Some(config_bytes) = self.get_plugin_configuration() {
            self.secret = String::from_utf8(config_bytes).unwrap()
        }
    }
}
```

```

}

true
}

fn create_http_context(&self, _: u32) -> Option<Box<dyn HttpContext>> {
    Some(Box::new(DemoPlugin {
        secret: self.secret.clone(),
    }))
}

fn get_type(&self) -> Option<ContextType> {
    Some(ContextType::HttpContext)
}

}

impl Context for DemoPluginRoot {}

pub fn parse_url_fullpath(url_string:&str,private_key:&str) -> Result<String,String>
{
    //split url to vec
    let vec: Vec<&str> = url_string.split("/").collect::<Vec<&str>>();
    //get fileid from url
    let filename=vec[vec.len()-1].split(".").collect::<Vec<&str>>();
    let fileid=filename[0];

```

```

info!("{}",format!("file is {}",fileid));

let presudo_url="http://localhost".to_string()+url_string;
info!("{}",format!("presudo_url is {}",presudo_url));
let parsed_url = Url::parse(&presudo_url).unwrap();
let hash_query: HashMap<_, _> = parsed_url.query_pairs().into_owned().collect();


if !hash_query.get("txSecret").is_some() {
return Err(format!("txSecret is missing"));
}

if !hash_query.get("txTime").is_some() {
return Err(format!("txTime is missing"));
}

info!("{}",format!("txSecret is {}",hash_query.get("txSecret").unwrap()));
info!("{}",format!("txTime is {}",hash_query.get("txTime").unwrap()));
let token=hash_query.get("txSecret").unwrap();


//get current timestamp
let now = SystemTime::now();
let since_the_epoch = now
.duration_since(UNIX_EPOCH)
.expect("Time went backwards").as_secs();

//get timestamp from url
let timestamp;
let timestamp_s=hash_query.get("txTime").unwrap();
let f=u64::from_str_radix(timestamp_s,16);
match f {
Ok(_v) => {timestamp=f.unwrap()}
Err(_f)=> {
return Err(format!("timestamp is invalid:{}",timestamp_s));
}
}

```

```

}
}

info!("{}",format!("get timestamp is {}",timestamp_s));
//check timestamp,if current is greater than timestamp,will panic
if since_the_epoch > (timestamp+86400) {
return Err(format!("timestamp is invalid:{}",timestamp_s))
}

//get real backend url and sign url from path
let mut sign_url=private_key.to_string();
sign_url+=&fileid.to_string();
sign_url+=&timestamp.to_string();
info!("{}",format!("sign value is {}",sign_url));
let digest =format!("{}",md5::compute(sign_url));
if token.eq(&digest) {
Ok(url_string.to_string())
} else {
return Err(format!("token is invalid:{},correct should be:{}",url_string,digest))
}
}

struct DemoPlugin {
secret: String,
}

impl HttpContext for DemoPlugin {
fn on_http_request_headers(&mut self, _: usize, _: bool) -> Action {
let http_path=self.get_http_request_header(":path").unwrap();

```

```

let result=parse_url_fullpath(&http_path,&self.secret);
match result {
Ok(v) => {
info!("Access granted.");
}
Err(v) => {
info!("{}",format!("return error:{}",v));
self.send_http_response(403, vec![], Some(b"Access forbidden.\n"));
return Action::Pause;
}
}

Action::Continue
}
}

impl Context for DemoPlugin {}

```

Cargo.toml

```

[package]
name = "my-wasm-plugin"
version = "0.1.0"
edition = "2021"
[dependencies]
proxy-wasm = "0.2"

```



```
log = "0.4"
md5= "0.7.0"
url= "2.3.1"
lazy_static="1.4.0"
regex="1.9.0"

[lib]
crate-type = ["cdylib"]
```

Deploy note

```
cargo build --release --target wasm32-wasi
docker build --no-cache --platform wasm -t tencent-wasm-plugin-md5-v4 .
docker tag tencent-wasm-plugin-md5-v4 \
asia-southeast1-docker.pkg.dev/jian-lin-aroglis-common/my-docker-repo/tencent-wasm-pl
ugin-md5-v4:v4
docker push \
asia-southeast1-docker.pkg.dev/jian-lin-aroglis-common/my-docker-repo/tencent-wasm-pl
ugin-md5-v4:v4

gcloud alpha service-extensions wasm-plugin-versions create
tencent-wasm-plugin-md5-v4-version-1 \
  --wasm-plugin=tencent-wasm-plugin-md5-v4 \

--image="asia-southeast1-docker.pkg.dev/jian-lin-aroglis-common/my-docker-repo/tencent
-wasm-plugin-md5-v4:v1" \
-plugin-config="mytestkey1|mytestkey2"

gcloud alpha service-extensions wasm-plugins update tencent-wasm-plugin-md5-v4 \
  --main-version=tencent-wasm-plugin-md5-v4-version-1

gcloud alpha service-extensions wasm-actions create tencent-md5-v4 \
  --wasm-plugin=tencent-wasm-plugin-md5-v4
```

```
gcloud edge-cache services export tencent-md5-v4 \
--destination=tencent-md5-v4.yaml
```

Add:

```
wasAction: "tencent-md5-v4"
```

In file "tencent-md5-v4.yaml"

```
gcloud alpha edge-cache services import tencent-md5-v4 \
--source=tencent-md5-v4.yaml
```

Final yaml file

```
logConfig:
  enable: true
  sampleRate: 1.0
name:
projects/jian-lin-aroglis-common/locations/global/edgeCacheServices/tencent-md5-v4
routing:
  hostRules:
    - hosts:
        - www.md5tencent.com
      pathMatcher: path-matcher-0
    - hosts:
        - www.md5mytest.com
      pathMatcher: path-matcher-1
  pathMatchers:
    - name: path-matcher-0
      routeRules:
        - headerAction: {}
      matchRules:
        - ignoreCase: true
          pathTemplateMatch: /**.m3u8
      origin: projects/251868301847/locations/global/edgeCacheOrigins/tencent-md5-v4
      priority: 10
      routeAction:
        cdnPolicy:
          cacheKeyPolicy: {}
          cacheMode: CACHE_ALL_STATIC
          defaultTtl: 3600s
          maxTtl: 86400s
```

```
    signedRequestMode: DISABLED
    wasmAction: "tencent-md5-v4"
    urlRewrite:
      hostRewrite: play-live-mojapp.tliveapp.com
- headerAction: {}
matchRules:
- prefixMatch: /
origin: projects/251868301847/locations/global/edgeCacheOrigins/tencent-md5-v4
priority: 11
routeAction:
  cdnPolicy:
    cacheKeyPolicy: {}
    cacheMode: FORCE_CACHE_ALL
    defaultTtl: 3600s
    signedRequestMode: DISABLED
    urlRewrite:
      hostRewrite: play-live-mojapp.tliveapp.com
- name: path-matcher-1
routeRules:
- headerAction: {}
matchRules:
- ignoreCase: true
  pathTemplateMatch: /**.m3u8
origin: projects/251868301847/locations/global/edgeCacheOrigins/my-orginal-1
priority: 10
routeAction:
  cdnPolicy:
    cacheKeyPolicy: {}
    cacheMode: CACHE_ALL_STATIC
    defaultTtl: 3600s
    maxTtl: 86400s
    signedRequestMode: DISABLED
    wasmAction: "tencent-md5-v4"
- headerAction: {}
matchRules:
- prefixMatch: /
origin: projects/251868301847/locations/global/edgeCacheOrigins/my-orginal-1
priority: 11
routeAction:
  cdnPolicy:
    cacheKeyPolicy: {}
    cacheMode: FORCE_CACHE_ALL
    defaultTtl: 3600s
    signedRequestMode: DISABLED
```

Test result :

curl

```
http://34.104.33.145/b423e0b1e544fe0e54b329c29c67734d/64A654C4/sample_480_640.m3u8
```

```
-H "host: www.md5mytest.com"
```

this is server

MD5 Version 6

Schema :

/txSecret/txTime/part_1/part_2/part_3/fileName

鉴权需求:

只有m3u8需要鉴权。

ts不鉴权。

url格式:

/txSecret/txTime/part_1/part_2/part_3/fileName

鉴权规则:

两个鉴权字段 txSecret , txTime 放到了url里面, 如:

/txSecret/txTime/part_1/part_2/part_3/fileName ,

例如;

https://xxxx.yyy.com/02f547f103a016b68e351a4eebe91240/768053ff/v1/0183ea205add0b8ed5941a38bc6f/0183ea20909b0b8ed5aa4d793456/ARYNewsH264_720p.m3u8

鉴权字段说明:

txTime: 和官网一致, 16进制秒级, 大小写都要支持, 有效范围是: txTime + 86400s。

txSecret: md5(key + /part_1/part_2/part_3 + txTime), 其中txTime不要做大小写转换, 用url里面原始的。

key: 为密钥

txSecret计算实例:

```
# printf "%s%s%s" adinw8m2wHyK4TDhiyTR
```

```
/v1/0183ea205add0b8ed5941a38bc6f/0183ea20909b0b8ed5aa4d793456 768053ff |  
md5sum  
02f547f103a016b68e351a4eebe91240 -
```

回源url改写:

回源的时候需要进行url重写去掉 txSecret , txTime 两个字段, 改写成
/part_1/part_2/part_3/fileName

m3u8 , ts 的url都需要去掉 txSecret , txTime 两个字段,然后再回源

例如:

m3u8 url

```
https://xxxx.yyy.com/02f547f103a016b68e351a4eebe91240/768053ff/v1/0183ea205add0b8e  
d5941a38bc6f/0183ea20909b0b8ed5aa4d793456/ARYNewsH264_720p.m3u8
```

```
https://xxxx.yyy.com/v1/0183ea205add0b8ed5941a38bc6f/0183ea20909b0b8ed5aa4d79345  
6/ARYNewsH264_720p.m3u8
```

ts url

```
https://xxxx.yyy.com/02f547f103a016b68e351a4eebe91240/768053ff/v1/0183ea205add0b8e  
d5941a38bc6f/0183ea20909b0b8ed5aa4d793456/ARYNewsH264_720p_1231.ts
```

```
https://xxxx.yyy.com/v1/0183ea205add0b8ed5941a38bc6f/0183ea20909b0b8ed5aa4d79345  
6/ARYNewsH264_720p_1231.ts
```

缓存key需求:

要去掉uri中的 /txSecret/txTime, 防止鉴权参数影响命中率。

Source code

```
use log::info;  
use proxy_wasm::traits::*;  
use proxy_wasm::types::*;  
use std::time::{SystemTime, UNIX_EPOCH};  
use md5;  
proxy_wasm::main! {{
```

```

proxy_wasm::set_log_level(LogLevel::Trace);

proxy_wasm::set_root_context(|_| -> Box<dyn RootContext> {
    Box::new(DemoPluginRoot {
        secret: String::new(),
    })
});
}}

struct DemoPluginRoot {
    secret: String,
}

impl RootContext for DemoPluginRoot {
    fn on_configure(&mut self, _: usize) -> bool {
        if let Some(config_bytes) = self.get_plugin_configuration() {
            self.secret = String::from_utf8(config_bytes).unwrap()
        }
        true
    }

    fn create_http_context(&self, _: u32) -> Option<Box<dyn HttpContext>> {
        Some(Box::new(DemoPlugin {
            secret: self.secret.clone(),
        })))
    }

    fn get_type(&self) -> Option<ContextType> {
        Some(ContextType::HttpContext)
    }
}

impl Context for DemoPluginRoot {}

struct DemoPlugin {

```

```

secret: String,
}

pub fn parse_url_fullpath(url_string:&str,private_key:&str) -> Result<String,String>
{
    //split url to vec
    let vec: Vec<&str> = url_string.split("/").collect::<Vec<&str>>();
    //get token from url
    let token=vec[1].to_string();
    info!("{}",format!("token is {}",token));
    //get fileid from url
    // let filename=vec[vec.len()-1].split(".").collect::<Vec<&str>>();
    // let fileid=filename[0];
    // info!("{}",format!("file is {}",fileid));
    let mut path="".to_string();
    for i in 3..vec.len()-1 {
        path=path+"/"+vec[i];
    }

    //get current timestamp
    let now = SystemTime::now();
    let current = now
        .duration_since(UNIX_EPOCH)
        .expect("Time went backwards").as_secs();
    //get timestamp from url
    let timestamp;
    let f=u64::from_str_radix(vec[2],16);
    match f {
        Ok(_v) => {timestamp=f.unwrap()}
        Err(_f)=> {

```

```

return Err(format!("timestamp is invalid in parse from hex to u64:{}",vec[2]));
}
}

info!("{}",format!("get timestamp is {}",vec[2]));
//check timestamp,if current is greater than timestamp then exist
//if since_the_epoch < timestamp {
// return Err(format!("timestamp exceed current timestamp
: {}, {}, {}",vec[2],since_the_epoch,timestamp))
// }

if current > (timestamp + 86400) {
return Err(format!("timestamp is expired : {}, {}, {}",vec[2],current,timestamp))
}

//get real backend url and sign url from path
let mut backend_url = "".to_string();
let sign_url = private_key.to_string() + &path + &vec[2];
for (index, el) in vec[0..].iter().enumerate() {
if index > 2 {
backend_url += &" / ".to_string();
backend_url += &el;
}
}

info!("{}",format!("backend url is {}",backend_url));
info!("{}",format!("sign value is {}",sign_url));
let digest = format!("{:x}",md5::compute(sign_url));
if token.to_lowercase().eq(&digest) {
Ok(backend_url.to_string())
} else {

```



```

return Err(format!("token is invalid: {}, correct should be: {}", url_string, digest))
}
}

impl HttpContext for DemoPlugin {
fn on_http_request_headers(&mut self, _: usize, _: bool) -> Action {
let http_path=self.get_http_request_header(":path").unwrap();
let result=parse_url_fullpath(&http_path,&self.secret);
match result {
Ok(v) => {
self.set_http_request_header(":path",Some(&v));
info!("Access granted.");
}
Err(v) => {
info!("{}",format!("return error:{}",v));
self.send_http_response(403, vec![], Some(b"Access forbidden.\n"));
return Action::Pause;
}
}
Action::Continue
}
}

impl Context for DemoPlugin {}

```

Deploy note

```
cargo build --release --target wasm32-wasi
```

```
docker build --no-cache --platform wasm -t tencent-wasm-plugin-md5-v6 .
docker tag tencent-wasm-plugin-md5-v6 \
asia-southeast1-docker.pkg.dev/jian-lin-aroglis-common/my-docker-repo/tencent-wasm-pl
ugin-md5-v6:latest
docker push \
asia-southeast1-docker.pkg.dev/jian-lin-aroglis-common/my-docker-repo/tencent-wasm-pl
ugin-md5-v6:latest
```

```
gcloud alpha service-extensions wasm-plugin-versions create
tencent-wasm-plugin-md5-v6 \
  --wasm-plugin=my-wasm-plugin-resource \
  --image="asia-southeast1-docker.pkg.dev/jian-lin-aroglis-common/my-docker-repo/tencent
-wasm-plugin-md5-v6:latest" \
  --plugin-config="adinw8m2wHyK4TDhiyTR"
```

```
gcloud alpha service-extensions wasm-plugins update my-wasm-plugin-resource \
  --main-version=tencent-wasm-plugin-md5-v6
```

```
gcloud alpha service-extensions wasm-actions create tencent-md5-v4 \
  --wasm-plugin=tencent-wasm-plugin-md5-v4
```

```
gcloud alpha edge-cache services export my-service \
  --destination=my-service.yaml
```

Add:

```
wasmAction: "tencent-md5-v4"
```

In file "tencent-md5-v4.yaml"

```
gcloud alpha edge-cache services importmy-service.yaml \
  --source=tencent-md5-v4.yaml
```

Header rewrite

Set http header "range" from http request query parameter "star","end" and also change the response code from 206 to 200.

```

use proxy_wasm::traits::*;
use proxy_wasm::types::*;
use url::Url;
use std::collections::HashMap;

proxy_wasm::main! {{
    proxy_wasm::set_log_level(LogLevel::Trace);
    proxy_wasm::set_root_context(|_| -> Box<dyn RootContext> { Box::new(HttpRoot) });
}}

struct HttpRoot;
impl Context for HttpRoot {}
impl RootContext for HttpRoot {
    fn get_type(&self) -> Option<ContextType> {
        Some(ContextType::HttpContext)
    }
    fn create_http_context(&self, _: u32) -> Option<Box<dyn HttpContext>> {
        Some(Box::new(TencentHttpContext {
            has_start_end: 0
        }))
    }
}

struct TencentHttpContext {
    has_start_end: u8,
}

impl Context for TencentHttpContext {}
impl HttpContext for TencentHttpContext {

```

```

fn on_http_request_headers(&mut self, _: usize, _: bool) -> Action {

    self.has_start_end=0;

    let http_path=self.get_http_request_header(":path").unwrap();
    let pseudo_url="http://localhost".to_string()+&http_path;
    let parsed_url = Url::parse(&pseudo_url).unwrap();
    let hash_query: HashMap<_, _> = parsed_url.query_pairs().into_owned().collect();

    if hash_query.get("start").is_none() ||
hash_query.get("start").unwrap().to_string().parse::<i32>().is_err() {
        return Action::Continue
    }

    if hash_query.get("end").is_none() ||
hash_query.get("end").unwrap().to_string().parse::<i32>().is_err() {
        return Action::Continue
    }

    let
range_header=format!("bytes={}-{}",hash_query.get("start").unwrap(),hash_query.get("end").u
nwrap());

    self.set_http_request_header("range",Some(range_header.as_str()));
    self.has_start_end=1;
    Action::Continue
}

fn on_http_response_headers(&mut self, _: usize, _: bool) -> Action {
    if self.has_start_end == 1 &&
self.get_http_response_header(":status").unwrap().eq_ignore_ascii_case("206")
{
        self.set_http_response_header(":status", Some(&"200"))
    }
}

```

```
}
```

```
Action::Continue
```

```
}
```

```
}
```

```
cargo build --release --target wasm32-wasi
```

```
docker build --no-cache --platform wasm -t tencent-wasm-plugin-parameter-range .
```

```
docker tag tencent-wasm-plugin-parameter-range \
asia-southeast1-docker.pkg.dev/jian-lin-aroglis-common/my-docker-repo/tencent-wasm-pl
ugin-parameter-range:latest
```

```
docker push \
asia-southeast1-docker.pkg.dev/jian-lin-aroglis-common/my-docker-repo/tencent-wasm-pl
ugin-parameter-range:latest
```

```
gcloud alpha service-extensions wasm-plugin-versions create version-parameter-range-7 \
--wasm-plugin=my-wasm-plugin-resource \
--image="asia-southeast1-docker.pkg.dev/jian-lin-aroglis-common/my-docker-repo/tencent
-wasm-plugin-parameter-range:latest"
```

```
gcloud alpha service-extensions wasm-plugins update my-wasm-plugin-resource \
--main-version=version-parameter-range-7
```

```
gcloud alpha service-extensions wasm-plugin-versions delete version-parameter-range-6 \
--wasm-plugin=my-wasm-plugin-resource
```

```
gcloud alpha service-extensions wasm-actions create tencent-md5-v4 \
--wasm-plugin=tencent-wasm-plugin-md5-v4
```

```
gcloud alpha edge-cache services export my-service \
--destination=my-service.yaml
```

Add:

```
wasmAction: "my-wasm-action-resource"
```

```
gcloud alpha edge-cache services import my-service.yaml \
--source=my-service.yaml
```

MD5 version7(path,support config,keep request url)

Data 2024-07-16

```
use log::info;
use md5;
use proxy_wasm::traits::*;
use proxy_wasm::types::*;
use std::time::{SystemTime, UNIX_EPOCH};
proxy_wasm::main! {{
proxy_wasm::set_log_level(LogLevel::Trace);

proxy_wasm::set_root_context(|_| -> Box<dyn RootContext> {
Box::new(DemoPluginRoot {
secret: String::new(),
}))
});
}}

struct DemoPluginRoot {
    secret: String,
}

impl RootContext for DemoPluginRoot {
    fn on_configure(&mut self, _: usize) -> bool {
        if let Some(config_bytes) = self.get_plugin_configuration() {
            self.secret = String::from_utf8(config_bytes).unwrap()
        }
    }
}
```

```

        true
    }

    fn create_http_context(&self, _: u32) -> Option<Box<dyn HttpContext>> {
        Some(Box::new(DemoPlugin {
            secret: self.secret.clone(),
        }))
    }

    fn get_type(&self) -> Option<ContextType> {
        Some(ContextType::HttpContext)
    }
}

impl Context for DemoPluginRoot {}

struct DemoPlugin {
    secret: String,
}

pub fn parse_url_fullpath(url_string: &str, private_key: &str) -> Result<String,
String> {
    //split url to vec
    let vec: Vec<&str> = url_string.split("/").collect::<Vec<&str>>();
    //get token from url
    let token = vec[1].to_string();
    //get current timestamp
    let now = SystemTime::now();
    let since_the_epoch = now
        .duration_since(UNIX_EPOCH)
        .expect("Time went backwards")
        .as_secs();
    //get timestamp from url

    let timestamp;

    let f = u64::from_str_radix(vec[2], 16);
    match f {
        Ok(_v) => timestamp = f.unwrap(),
        Err(_f) => {
            return Err(format!(
                "timestamp is invalid in parse from hex to u64:{}",
                vec[2]
            ));
        }
    }
}

```

```

info!("{}", format!("get timestamp is {}", timestamp));
//check timestamp,if current is greater than timestamp,will panic
if since_the_epoch > timestamp {
    return Err(format!("timestamp is invalid:{}", vec[2]));
}

//get real backend url and sign url from path

// let mut backend_url = "".to_string();
let mut sign_url = private_key.to_string();
/*for (index, el) in vec[0..].iter().enumerate() {
    if index > 2 {
        backend_url += &"/".to_string();
        backend_url += &el;

        if index > 2 && index < vec.len() - 1 {
            sign_url += &"/".to_string();
            sign_url += &el;
        }
    }
}*/

sign_url += &vec[2].to_string();
//info!("{}", format!("backend url is {}", backend_url));
info!("{}", format!("sign value is {}", sign_url));

let digest = format!("{:x}", md5::compute(sign_url));
if token.eq(&digest) {
    Ok(String::from("Ok"))
} else {
    return Err(format!(
        "token {} is invalid:{},correct should be:{}",
        token, url_string, digest
    ));
}
}

impl HttpContext for DemoPlugin {
    fn on_http_request_headers(&mut self, _: usize, _: bool) -> Action {
        let http_path = self.get_http_request_header(":path").unwrap();
        let result = parse_url_fullpath(&http_path, &self.secret);
    }
}

```



```

        //let result = parse_url_fullpath(&http_path,
        &"e12c46f261245106e2554781ab261aa3");

        match result {
            Ok(v) => {
                info!("Access granted.");
            }
            Err(v) => {
                info!("{}", format!("return error:{}", v));
                self.send_http_response(403, vec![], Some(b"Access forbidden.\n"));
                return Action::Pause;
            }
        }
        Action::Continue
    }
}

impl Context for DemoPlugin {}

```

AWS sign url/Cookie

Origin version

```

use log::{debug};
use proxy_wasm::traits::*;
use proxy_wasm::types::*;

use base64ct::{Base64, Encoding};
use ipnetwork::IpNetwork;
use json;
use rsa::pkcs1v15::{Signature, VerifyingKey};
use rsa::pkcs8::DecodePublicKey;
use rsa::signature::Verifier;
use rsa::RsaPublicKey;
use sha1::Sha1;

```

```

use std::collections::HashMap;
use std::net::IpAddr;
use std::str::FromStr;
use std::time::{SystemTime, UNIX_EPOCH};
use thiserror::Error;
use url::Url;
use wildmatch::WildMatch;

#[derive(Error, Debug)]
pub enum EncodingError {
    #[error("invalid key provided")]
    InvalidKeyError(#[from] rsa::pkcs8::spki::Error),

    #[error("Url parse error")]
    UrlParseError(#[from] url::ParseError),

    #[error("int parse error")]
    IntParseError(#[from] std::num::ParseIntError),

    #[error("Base64 decode error")]
    Base64DecodeError(#[from] base64::DecodeError),

    #[error("Json Parse error")]
    JsonParseError(#[from] json::Error),

    #[error("String from Vec Parse error")]
    StringParseError(#[from] std::string::FromUtf8Error),

    #[error("IP Parse error")]
    IPParseError(#[from] std::net::AddrParseError),

    #[error("IP Network Parse error")]
    IPNetworkParseError(#[from] ipnetwork::IpNetworkError),

    #[error("unknown error")]
    Unknown,
}

struct SignedOptions {
    pub date_less_than: u64, // The expiration date and time for the URL in Unix time
                             // format (in seconds) and Coordinated Universal Time (UTC). Defaults to 1800s from

```

```

now.

    pub date_greater_than: Option<u64>, // An optional start date and time for the
URL in Unix time format (in seconds) and Coordinated Universal Time (UTC).
    pub ip_address: Option<String>, // An optional IP address of the client making
the GET request (can be a range).
}

impl Default for SignedOptions {
    fn default() -> SignedOptions {
        let since_epoch = SystemTime::now().duration_since(UNIX_EPOCH).unwrap();
        SignedOptions {
            date_less_than: since_epoch.as_secs() + 1800,
            date_greater_than: None,
            ip_address: None,
        }
    }
}

fn get_custom_policy(url: &str, options: &SignedOptions) -> String {
    let date_greater_than = options
        .date_greater_than
        .map(|date| format!("{}", "DateGreaterThan":{{"AWS:EpochTime\":"{:?}}}}, date))
        .unwrap_or_default();

    let ip_address = options
        .ip_address
        .clone()
        .map(|ip_address| format!("{}", "IpAddress\":"{{"AWS:SourceIp\":"{:?}}}},
ip_address))
        .unwrap_or_default();

    format!(
        "{{\"Statement\":[{{\"Resource\":"{{\"{{\"Condition\":"{{\"DateLessThan\":"{{\"AWS:Epo
chTime\":"{:?}}}}}}}}}}}}",
            url, options.date_less_than, date_greater_than, ip_address
        )
    }
}

fn to_normalize_base64(input: &str) -> String {
    input.replace('-', '+').replace('_', '=').replace('~', '/')
}

```

```

}

proxy_wasm::main! {{
proxy_wasm::set_log_level(LogLevel::Trace);
proxy_wasm::set_root_context(|_| -> Box<dyn RootContext> {
Box::new(DemoPluginRoot {
config: HashMap::new(),
})
});
}}

fn check_policy(

    policy_str: &str,
    request_url: &str,
    ip: &str,

) -> Result<bool, EncodingError> {
    debug!("the policy is \n{}\n", policy_str);
    let since_the_epoch = SystemTime::now()
        .duration_since(UNIX_EPOCH)
        .expect("Time went backwards")
        .as_secs();

    let parsed_policy: json::JsonValue = json::parse(policy_str)?;

    let request_policy_date_less_than =

parsed_policy["Statement"][0]["Condition"]["DateLessThan"]["AWS:EpochTime"].as_u64()
;

    if request_policy_date_less_than.is_some() {
        if since_the_epoch > request_policy_date_less_than.unwrap() {
            debug!("expire in policy is out of date\n");
            return Ok(false);
        }
    } else {
        debug!("expire in policy is missing\n");
        return Ok(false);
    }

    let request_policy_date_greater_than =

```

```

parsed_policy["Statement"][0]["Condition"]["DateGreaterThan"]["AWS:EpochTime"].as_u64();

    if request_policy_date_greater_than.is_some() {
        if since_the_epoch < request_policy_date_greater_than.unwrap() {
            debug!(
                "the date greater than in policy is less the system datetime {}",
                since_the_epoch,
                request_policy_date_greater_than.unwrap()
            );
            return Ok(false);
        }
    }

    let request_policy_source_address =
parsed_policy["Statement"][0]["Condition"]["IpAddress"]["AWS:SourceIp"].as_str();

    if request_policy_source_address.is_some() {
        // Parse the IP address and subnet
        let ip_address = IpAddr::from_str(ip)?;
        let subnet = IpNetwork::from_str(request_policy_source_address.unwrap())?;
        debug!(
            "ip address in xff:{}, in policy resource:{}",
            ip,
            request_policy_source_address.unwrap()
        );
        // Check if the IP address is within the subnet
        if !subnet.contains(ip_address) {
            return Ok(false);
        } else {
            debug!(
                "ip check ok, {} in {}",
                ip,
                request_policy_source_address.unwrap()
            )
        }
    }

    let request_policy_resouce = parsed_policy["Statement"][0]["Resource"].as_str();

```

```

    if request_policy_resource.is_some() {
        if !WildMatch::new(request_policy_resource.unwrap()).matches(request_url) {
            return Ok(false);
        }
    }

    return Ok(true);
}

fn verify_cookie(
    url: &str,
    public_key: &HashMap<String, String>,
    xff_addr: &str,
    cookie: HashMap<String, String>,
) -> Result<bool, EncodingError> {
    let mut sign_text = String::new();
    if (!cookie.contains_key("CloudFront-Signature"))
        || (!cookie.contains_key("CloudFront-Key-Pair-Id"))
        || ((!cookie.contains_key("CloudFront-Expires"))
            && (!cookie.contains_key("CloudFront-Policy")))
    {
        debug!("missing required cookie");
        return Ok(false);
    }
    debug!("signed cookie check start");

    if !public_key.contains_key(cookie.get("CloudFront-Key-Pair-Id").unwrap())
    {
        debug!("the key id is miss in config");
        return Ok(false);
    }

    if cookie.contains_key("CloudFront-Policy") {
        let policy = base64::decode(to_normalize_base64(
            cookie.get("CloudFront-Policy").unwrap(),
        ))?;

        let policy_str = String::from_utf8(policy)?;
        let policy_check_result = check_policy(policy_str.as_str(), url, xff_addr);
    }
}

```

```

match policy_check_result {
    Ok(v) => {
        if !v {
            debug!("policy_check_result failed");

            return Ok(false);
        }
    }
    Err(_m) => {
        debug!("policy check error");
        return Ok(false);
    }
}

sign_text = policy_str;
debug!("Sign Cookie ,the text to be verified is :{}", sign_text);

let signature =
to_normalize_base64(cookie.get("CloudFront-Signature").unwrap());

let sig = Base64::decode_vec(signature.as_str());

match sig {
    Ok(v) => {
        let public_key = RsaPublicKey::from_public_key_pem(
            public_key
                .get(cookie.get("CloudFront-Key-Pair-Id").unwrap())
                .clone()
                .unwrap(),
        )?;
        let verifying_key = VerifyingKey::<Shal>::new(public_key);
        let result = verifying_key.verify(
            sign_text.as_bytes(),
            &Signature::try_from(v.as_slice()).unwrap(),
        );
        match result {
            Ok(_v) => {
                debug!("cookie signature check passed");
                return Ok(true);
            }
            Err(_m) => {
                debug!("cookie signature check failed");
            }
        }
    }
}

```

```

        return Ok(false);
    }
}

Err(_m)=> {
    debug!("signature decode error");
    return Ok(false);
}

}

}

let since_the_epoch = SystemTime::now()
    .duration_since(UNIX_EPOCH)
    .expect("Time went backwards")
    .as_secs();

let request_expire = cookie.get("CloudFront-Expires").unwrap().parse::<u64>()?;
if since_the_epoch > request_expire {
    debug!("{}", "the sign has expired");
    return Ok(false);
}

if sign_text.len() == 0 {
    let options_from_url: SignedOptions = SignedOptions {
        date_less_than: request_expire,
        ..Default::default()
    };

    sign_text = get_custom_policy(url, &options_from_url);
}

debug!("Sign Cookie ,the text to be verified is :{}", sign_text);

let signature = to_normalize_base64(cookie.get("CloudFront-Signature").unwrap());

let sig = Base64::decode_vec(signature.as_str());

match sig {
    Ok(v) => {
        let public_key = RsaPublicKey::from_public_key_pem(
            public_key
                .get(cookie.get("CloudFront-Key-Pair-Id").unwrap())

```



```

        .clone()
        .unwrap(),
    )?;
    let verifying_key = VerifyingKey::<Sha1>::new(public_key);
    let result = verifying_key.verify(
        sign_text.as_bytes(),
        &Signature::try_from(v.as_slice()).unwrap(),
    );
    match result {
        Ok(_v) => {
            debug!("cookie signature check passed");
            return Ok(true);
        }
        Err(_m) => {
            debug!("cookie signature check failed");
            return Ok(false);
        }
    }
}

Err(_m) => {
    debug!("signature decode error");
    return Ok(false);
}
}

fn verify_url(
    signed_url: &str,
    public_key: &HashMap<String, String>,
    xff_addr: &str,
) -> Result<bool, EncodingError> {
    debug!("the url is: {}", signed_url);
    let mut sign_text = String::new();
    let mut parsed_url = Url::parse(&signed_url)?;
    let hash_query: HashMap<_, _> = parsed_url.query_pairs().into_owned().collect();

    let query: Vec<(String, String)> = parsed_url
        .query_pairs()
        .filter(|(name, _)| name != "Expires")
        .filter(|(name, _)| name != "Policy")
        .filter(|(name, _)| name != "Signature")

```

```

        .filter(|(name, _) | name != "Key-Pair-Id")
        .map(|(name, value)| (name.into_owned(), value.into_owned()))
        .collect();

parsed_url.query_pairs_mut().clear().extend_pairs(&query);

let size = query.len();
if size > 0 {
    parsed_url.query_pairs_mut().clear().extend_pairs(query);
} else {
    parsed_url.set_query(None);
}

let request_policy: Vec<u8>;

let request_url = parsed_url.as_str();
debug!(
    "the resouce url generated from request is:{}\n",
    request_url
);

if ((!hash_query.contains_key("Expires")) &&
(!hash_query.contains_key("Policy")))
    || (!hash_query.contains_key("Signature"))
    || (!hash_query.contains_key("Key-Pair-Id"))
{
    debug!("missing required parameter in query");
    return Ok(false);
}

let request_key_pair_id = hash_query.get("Key-Pair-Id").unwrap().as_str();

if !public_key.contains_key(request_key_pair_id)
{
    debug!("the key id is miss in config");
    return Ok(false);
}

let request_signature = hash_query.get("Signature").unwrap().as_str();

// check custom policy

```

```

if hash_query.get("Policy").is_some() {
    debug!("using policy sign:{}", hash_query.get("Policy").unwrap());

    request_policy =
base64::decode(to_normalize_base64(hash_query.get("Policy").unwrap()))?;
    sign_text = String::from_utf8(request_policy)?;

    //check policy start

    let policy_check_result = check_policy(sign_text.as_str(), request_url,
xff_addr);
    //check policy end
    match policy_check_result {
        Ok(v) => {
            if !v {
                debug!("policy content check failed");
                return Ok(false);
            } else {
                debug!("the text to be verified is : {}", sign_text);
                let signature = to_normalize_base64(request_signature);
                let sig = Base64::decode_vec(signature.as_str());

                match sig {
                    Ok(v) => {
                        let public_key = RsaPublicKey::from_public_key_pem(
                            public_key.get(request_key_pair_id).clone().unwrap(),
                        )?;
                        let verifying_key =
VerifyingKey::<Sha1>::new(public_key);
                        let result = verifying_key.verify(
                            sign_text.as_bytes(),
                            &Signature::try_from(v.as_slice()).unwrap(),
                        );
                        match result {
                            Ok(_v) => return Ok(true),
                            Err(_m) => return Ok(false),
                        }
                    }
                    Err(_m) => return Ok(false),
                }
            }
        }
    }
}

```

```

    }
}
Err(_m) => {
    debug!("policy decode error");
    return Ok(false);
}
}

//check canned policy

let request_expire = hash_query.get("Expires").unwrap().parse::<u64>()?;
let since_the_epoch = SystemTime::now()
    .duration_since(UNIX_EPOCH)
    .expect("Time went backwards")
    .as_secs();
if since_the_epoch > request_expire {
    debug!("{}", "the sign has expired");
    return Ok(false);
}

if sign_text.len() == 0 {
    let options_from_url: SignedOptions = SignedOptions {
        date_less_than: request_expire,
        ..Default::default()
    };

    sign_text = get_custom_policy(request_url, &options_from_url);
}
debug!("the text to be verified is : {}", sign_text);
let signature = to_normalize_base64(request_signature);
let sig = Base64::decode_vec(signature.as_str());

match sig {
    Ok(v) => {
        let public_key = RsaPublicKey::from_public_key_pem(
            public_key.get(request_key_pair_id).clone().unwrap(),
        )?;
        let verifying_key = VerifyingKey::<Shal>::new(public_key);
        let result = verifying_key.verify(
            sign_text.as_bytes(),
            &Signature::try_from(v.as_slice()).unwrap(),
        );
    }
}

```

```

    );

    match result {
        Ok(_v) => return Ok(true),
        Err(_m) => return Ok(false),
    }

    }

    Err(_m) => return Ok(false),
}

}

struct DemoPluginRoot {
    config: HashMap<String, String>,
}

impl RootContext for DemoPluginRoot {
    fn on_configure(&mut self, _: usize) -> bool {
        if let Some(config_bytes) = self.get_plugin_configuration() {
            let config_str = String::from_utf8(config_bytes).unwrap();
            let configs: Vec<&str> = config_str.split(";").collect();
            for config_itme in configs {
                let parts: Vec<&str> = config_itme.splitn(2, "=").collect();
                if parts.len() == 2 {
                    let name = parts[0].trim().to_string();
                    let value = parts[1].trim().to_string();
                    self.config.insert(name, value);
                }
            }
        }

        true
    }

    fn create_http_context(&self, _: u32) -> Option<Box<dyn HttpContext>> {
        Some(Box::new(DemoPlugin {
            config: self.config.clone(),
        }))
    }

    fn get_type(&self) -> Option<ContextType> {
        Some(ContextType::HttpContext)
    }
}

```

```

impl Context for DemoPluginRoot {}

struct DemoPlugin {
    config: HashMap<String, String>,
}

impl HttpContext for DemoPlugin {
    fn on_http_request_headers(&mut self, _: usize, _: bool) -> Action {
        //let http_host = self.config.get("host").unwrap().clone();
        let http_host = self.get_http_request_header(":authority").unwrap();
        let http_scheme = self.get_http_request_header(":scheme").unwrap();
        let http_path = self.get_http_request_header(":path").unwrap();
        let http_xff = self.get_http_request_header("x-forwarded-for").unwrap();
        let addr = http_xff.as_str();
        debug!(
            "the request is {}://{{{}}}, from ip address {}",
            http_scheme, http_host, http_path, addr
        );

        //self.config.remove("host");

        // sign url validation
        let signed_url = format!("{}://{{{}}}", http_scheme, http_host, http_path);

        debug!("checking sign url");
        let result = verify_url(signed_url.as_str(), &self.config, addr);

        match result {
            Ok(v) => {
                if v {
                    debug!("signed url check is ok");
                    return Action::Continue;
                } else {
                    debug!("signed url check failed due to signature not match");
                }
            }
            Err(msg) => {
                debug!("sign url check failed on:{}", msg);
            }
        }
    }
}

```

```

debug!("checking cookie");

let mut cookie_map: HashMap<String, String> = HashMap::new();
if let Some(cookie_header) = self.get_http_request_header("Cookie") {
    let cookies: Vec<&str> = cookie_header.split(";").collect();
    for cookie in cookies {
        let parts: Vec<&str> = cookie.splitn(2, "=").collect();
        if parts.len() == 2 {
            let name = parts[0].trim().to_string();
            let value = parts[1].trim().to_string();
            debug!("find cookie:{}={}", name, value);
            cookie_map.insert(name, value);
        }
    }

    let result = verify_cookie(signed_url.as_str(), &self.config, addr,
cookie_map);

    match result {
        Ok(v) => {
            if v {
                debug!("signed cookie check pass");
                return Action::Continue;
            } else {
                self.send_http_response(403, vec![], Some(b"Access
forbidden.\n"));

                return Action::Pause;
            }
        }
        Err(msg) => {
            debug!("verify cookie failed on:{}",msg);
            self.send_http_response(403, vec![], Some(b"Access
forbidden.\n"));

            return Action::Pause;
        }
    }

    self.send_http_response(403, vec![], Some(b"Access forbidden.\n"));
    return Action::Pause;
}
}

```

```
impl Context for DemoPlugin {}
```

```
[package]
name = "my-wasm-plugin"
version = "0.1.0"
edition = "2021"
[dependencies]
proxy-wasm = "0.2"
log = "0.4"

base64 = "0.12.3"
url = "2.5.0"
json = "0.12.4"
rsa = { version = "0.9.6", features = ["sha2","sha1"] }
base64ct = "1.6.0"
sha1 = "0.10.6"
thiserror = "1.0.56"
ipnetwork = "0.20.0"

[lib]
crate-type = ["cdylib"]
```

Deployment step(turn to shell script)

```
export ver=`cat /proc/sys/kernel/random/uuid`
```



```
export plugin='aws-sign'
echo $plugin-$ver
cp ./Cargo.toml ./Cargo.toml.exec
rm ./Cargo.toml
cp ./Cargo.toml.wasm ./Cargo.toml
rm ./target/wasm32-wasi/release/my_wasm_plugin.wasm

cargo build --release --target wasm32-wasi
docker build --no-cache --platform wasm -t $plugin .
docker tag aws-sign \
asia-southeast1-docker.pkg.dev/jian-lin-aroglis-common/my-docker-repo/$plugin:latest
docker push \
asia-southeast1-docker.pkg.dev/jian-lin-aroglis-common/my-docker-repo/$plugin:latest

gcloud alpha service-extensions wasm-plugin-versions create $plugin-$ver \
--wasm-plugin=$plugin-plugin \
--image="asia-southeast1-docker.pkg.dev/jian-lin-aroglis-common/my-docker-repo/$plugin:latest" \
--plugin-config="key1-----BEGIN PUBLIC KEY-----
MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEAlRhZ4JcKBax9j1V4UVAk
ARXGzIPFtfTwOdHrJhrcRY40370BQHSwiQC7qX8Jx+6v+VrGXDQQh0h4N5gVKI1G
EtqcE5415Jv9NvcmBlyG6vJWSGWCXBbYmoKig1a+3OucB7/n6wD4DCNBzq9gHdIO
D1C32AulMb5HomaPKnw+bIYaxIBBpI4gSwGsFCLamPQHqpaszEgtz7Tta9W0gf9v
55Log8n7PG4RfcSIOo38ar+PSLOxL3oIbJqEZPw9/0ldkm9jAwf4tVuigfW/33QE
S2V5qoVtGbbZVaxGYtnPQYDnyBd0c11xnAiL0tppsH6EwMvL40gw4wLJnULHDHOM
EwIDAQAB
-----END PUBLIC KEY-----;key2-----BEGIN PUBLIC KEY-----
MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEAvm4MNahSSbIDKBTAsox
S1lDgE4FgPpc0rdmmgnk15qcWLumfliLnzkb9jZLB4wVduRLZdRfc/p1k16VZKug
6gr24eHtIqbpZY+enA7UGqmxm6BIsCHUCjD3BhZ6BvH631S+5cG4qppJEw6801cK
n7iecyZq38zn4iT2Xwf5Y43VQMUDtv7r9LEJIWLVza+QFlkAsAmORHwnPoKRIPC
j/qIgKdX6H/cmHW2KgbEYDNiyN1EV9NIR4Ym6d2rX2f7530iWY7PaUB8LqhTpbTG
pgR2Ca5CQGJHZN06VbMe8fyL091cV41iGY4M32SVugdIHgpF2/qVb2y8RnGs4asY
aQIDAQAB
-----END PUBLIC KEY-----;key3-----BEGIN PUBLIC KEY-----
MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEAlRhZ4JcKBax9j1V4UVAk
ARXGzIPFtfTwOdHrJhrcRY40370BQHSwiQC7qX8Jx+6v+VrGXDQQh0h4N5gVKI1G
EtqcE5415Jv9NvcmBlyG6vJWSGWCXBbYmoKig1a+3OucB7/n6wD4DCNBzq9gHdIO
D1C32AulMb5HomaPKnw+bIYaxIBBpI4gSwGsFCLamPQHqpaszEgtz7Tta9W0gf9v
55Log8n7PG4RfcSIOo38ar+PSLOxL3oIbJqEZPw9/0ldkm9jAwf4tVuigfW/33QE
```

S2V5qoVtGbbZVaxGYtnPQYDnyBd0c1lxnAiL0tppsH6EwMvL40gw4wLJnULHDH0M
EwIDAQAB

-----END PUBLIC KEY-----;key4-----BEGIN PUBLIC KEY-----

MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEAlRhZ4JcKBax9j1V4UVAK
ARXGzIPFtfTwOdHrJhrcRY4O370BQHSwiQC7qX8Jx+6v+VrGXDQQh0h4N5gVKI1G
EtqcE5415Jv9NvcmBlyG6vJWSGWCXBbYmoKig1a+3OucB7/n6wD4DCNBzq9gHdIO
D1C32AulMb5HomaPKnw+bIYaxIBBpI4gSwGsFCLamPQHQpaszEgtz7Tta9W0gf9v
55Log8n7PG4RfcSIOo38ar+PSLOxL3oIbJqEzPw9/0ldkm9jAwf4tVuigfW/33QE
S2V5qoVtGbbZVaxGYtnPQYDnyBd0c1lxnAiL0tppsH6EwMvL40gw4wLJnULHDH0M
EwIDAQAB

-----END PUBLIC KEY-----;key5-----BEGIN PUBLIC KEY-----

MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEAlRhZ4JcKBax9j1V4UVAK
ARXGzIPFtfTwOdHrJhrcRY4O370BQHSwiQC7qX8Jx+6v+VrGXDQQh0h4N5gVKI1G
EtqcE5415Jv9NvcmBlyG6vJWSGWCXBbYmoKig1a+3OucB7/n6wD4DCNBzq9gHdIO
D1C32AulMb5HomaPKnw+bIYaxIBBpI4gSwGsFCLamPQHQpaszEgtz7Tta9W0gf9v
55Log8n7PG4RfcSIOo38ar+PSLOxL3oIbJqEzPw9/0ldkm9jAwf4tVuigfW/33QE
S2V5qoVtGbbZVaxGYtnPQYDnyBd0c1lxnAiL0tppsH6EwMvL40gw4wLJnULHDH0M
EwIDAQAB

-----END PUBLIC KEY-----;key6-----BEGIN PUBLIC KEY-----

MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEAlRhZ4JcKBax9j1V4UVAK
ARXGzIPFtfTwOdHrJhrcRY4O370BQHSwiQC7qX8Jx+6v+VrGXDQQh0h4N5gVKI1G
EtqcE5415Jv9NvcmBlyG6vJWSGWCXBbYmoKig1a+3OucB7/n6wD4DCNBzq9gHdIO
D1C32AulMb5HomaPKnw+bIYaxIBBpI4gSwGsFCLamPQHQpaszEgtz7Tta9W0gf9v
55Log8n7PG4RfcSIOo38ar+PSLOxL3oIbJqEzPw9/0ldkm9jAwf4tVuigfW/33QE
S2V5qoVtGbbZVaxGYtnPQYDnyBd0c1lxnAiL0tppsH6EwMvL40gw4wLJnULHDH0M
EwIDAQAB

-----END PUBLIC KEY-----;key7-----BEGIN PUBLIC KEY-----

MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEAlRhZ4JcKBax9j1V4UVAK
ARXGzIPFtfTwOdHrJhrcRY4O370BQHSwiQC7qX8Jx+6v+VrGXDQQh0h4N5gVKI1G
EtqcE5415Jv9NvcmBlyG6vJWSGWCXBbYmoKig1a+3OucB7/n6wD4DCNBzq9gHdIO
D1C32AulMb5HomaPKnw+bIYaxIBBpI4gSwGsFCLamPQHQpaszEgtz7Tta9W0gf9v
55Log8n7PG4RfcSIOo38ar+PSLOxL3oIbJqEzPw9/0ldkm9jAwf4tVuigfW/33QE
S2V5qoVtGbbZVaxGYtnPQYDnyBd0c1lxnAiL0tppsH6EwMvL40gw4wLJnULHDH0M
EwIDAQAB

-----END PUBLIC KEY-----;key8-----BEGIN PUBLIC KEY-----

MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEAlRhZ4JcKBax9j1V4UVAK
ARXGzIPFtfTwOdHrJhrcRY4O370BQHSwiQC7qX8Jx+6v+VrGXDQQh0h4N5gVKI1G
EtqcE5415Jv9NvcmBlyG6vJWSGWCXBbYmoKig1a+3OucB7/n6wD4DCNBzq9gHdIO
D1C32AulMb5HomaPKnw+bIYaxIBBpI4gSwGsFCLamPQHQpaszEgtz7Tta9W0gf9v
55Log8n7PG4RfcSIOo38ar+PSLOxL3oIbJqEzPw9/0ldkm9jAwf4tVuigfW/33QE
S2V5qoVtGbbZVaxGYtnPQYDnyBd0c1lxnAiL0tppsH6EwMvL40gw4wLJnULHDH0M

```
EwIDAQAB
-----END PUBLIC KEY-----;host=test.com"

gcloud alpha service-extensions wasm-plugins update $plugin-plugin \
--main-version=$plugin-ver

gcloud alpha service-extensions wasm-plugin-versions list
--wasm-plugin=$plugin-plugin|awk -F" " '{IF NF>1 print$1}'
```

```
gcloud edge-cache services export my-cme \
--destination=my-cme.yaml
```

Add:

```
wasmAction: "aws-sign-action"
```

In file "tencent-md5-v4.yaml"

```
gcloud alpha edge-cache services import my-cme \
--source=my-cme.yaml
```

Enable Logging

```
gcloud alpha service-extensions wasm-plugins update aws-sign-plugin \
--log-config=enable=True,sample-rate=1.0,min-log-level=INFO
```

Test command

Sign url custom policy

policy

```
{
  "Statement": [
    {
      "Resource": "http://*",
      "Condition": {
        "DateLessThan": {
          "AWS:EpochTime": 1709106004
        },
        "DateGreaterThan": {
          "AWS:EpochTime": 1706028560
        },
        "IpAddress": {
          "AWS:SourceIp": "0.0.0.0/0"
        }
      }
    }
  ]
}
```

```
curl
'http://34.104.37.211/sample_1m.txt?Expires=1707106004&Policy=eyJTdGF0ZWl1bnQiOlt7IlJlc
291cmNlIjoikjovLyoLCJDb25kaXRpb24iOnsiRGF0ZUxlc3NUaGFuIjpb7IkFXUzpFcG9jaFRpbWUi
OjE3MDcxMDYwMDR5LCJJcEFkZHZHJlc3MiOnsiQVdTOiNvdXJjZUlwIjoiMC4wLjAuMC8wLn19fV
19&Signature=il7ZE5x-l1FyEVOTygTXagklQ6F~hNrZ7NRe-jl~X53Xycs8qkeg3Be-XeRvq-iWaN7
HrYvgI0nit0DOUQY1L6zwZdggErVDwTyymxgmMootz0quiAeS2uylA~6sR4mvOusm-DzTswul
B~H11cgEiTyhvfUJ8sUC81OEPIQeKy6bLoZzcX7wT2fDDawoQcV~6t~UgFbSxWLDvffN-Ao~J1z
asQqhog21zvJmTbOIQ8PdyF9jRGcuGnXPLyyWjg12mpmUtl8GQ-EL5v8k17FDKf4MF81w2ld4h
nqnMS2HTCqTT5~1PRTJqZq7ov6zyJvBtTn7-ZZG211Oa4NIACxfJA__&Key-Pair-Id=key2' -H
'host: test.com'
```

[Sign cookie custom policy](#)

```
curl 'http://34.104.37.211/sample_1m.txt' -H "Cookie:
CloudFront-Signature=il7ZE5x-l1FyEVOTygTXagklQ6F~hNrZ7NRe-jl~X53Xycs8qkeg3Be-XeRv
q-iWaN7HrYvgI0nit0DOUQY1L6zwZdggErVDwTyymxgmMootz0quiAeS2uylA~6sR4mvOusm
-DzTswulB~H11cgEiTyhvfUJ8sUC81OEPIQEkY6bLoZzc7wT2fDDawoQcV-6t~UgFbSxWLdvff
N-Ao~J1zasQqhog21zvJmtbOIQ8PdyF9jRGcuGnXPLyyWjg12mpmUtl8GQ-EL5v8k17FDKf4M
F81w2ld4hnqnMS2HTCqTT5~1PRTJqZq7ov6zyJvBtTn7-ZZG2I1Oa4NIAcXfJA__";
CloudFront-Key-Pair-Id=key2; CloudFront-Expires=1707106004;
CloudFront-Policy=eyJTdGF0ZW1lbnQiOiIt7lJlc291cmNlljoiKjovLyoiLCJDb25kaXRpb24iOnsiR
GF0ZUxlczNUaG9Fuljp7lklFXUzpfFcG9jaFRpbWUiOiJEMDcxMDYwMDR9LCJJeEFkZlJlc3MiOn
siQVdTOiNvdXJjZUlwljoiMC4wLjAuMC8wLn19fV19" -H "host: test.com"
```

Sign cookie default policy

```
curl 'http://34.104.37.211/sample_1m.txt?test=1' -H "Cookie:
CloudFront-Signature=VD1VRaVGKGtE~5Cbu1j~7b2LapPSZRyoUrvUo-4KCH4zSuMCmK~njiE
2xrFXkaqOY2U1my9tszTCAQaKhq65-IRkmmYxIZM3H0-wPULvNkokV1omqx7btPpXNIMcFm5
```

```
Ftx9SPPhnblZdJJL3UsBrDFsS8cMO8odDY4MfQCnOFdxnDDzvYxiX5erv0AJB3NSRC-cVZKKBxJ726B-EqD9IVx8LIAID63L2wzacIZ0Z43zo46WjnVajDDP2Nq56ZiBnNnf-KWA-q3lx7lnNn34eysloNzVuMsnVmSCn3xdoCmNMVXMcON~9kqwIDy~JbKDs~bFCFvqaJMVBUyZlz6NUog__;  
CloudFront-Key-Pair-Id=key2; CloudFront-Expires=1707106004" -H "host: test.com"
```

Sign url default policy

```
curl  
'http://34.104.37.211/sample_1m.txt?test=1&Expires=1707106004&Signature=VD1VRaVGKGtE~5Cbu1j~7b2LapPSZRyoUrvUo~4KCH4zSuMCMK~njiE2xrFXkaqOY2U1my9tszTCAQaKhq65-IRkmmYxlZM3H0~wPULvNkokV1omqx7btPpXNIMcFm5Ftx9SPPhnblZdJJL3UsBrDFsS8cMO8odDY4MfQCnOFdxnDDzvYxiX5erv0AJB3NSRC-cVZKKBxJ726B-EqD9IVx8LIAID63L2wzacIZ0Z43zo46WjnVajDDP2Nq56ZiBnNnf-KWA-q3lx7lnNn34eysloNzVuMsnVmSCn3xdoCmNMVXMcON~9kqwIDy~JbKDs~bFCFvqaJMVBUyZlz6NUog__&Key-Pair-Id=key2' -H "host: test.com"
```

Lazy_static with Rwlock version

```
/*  
update on 00:01 Mar 02  
*/  
use log::debug;  
use proxy_wasm::traits::*;  
use proxy_wasm::types::*;  
  
use base64ct::{Base64, Encoding};  
use ipnetwork::IpNetwork;  
use json;  
use lazy_static::lazy_static;  
use rsa::pkcs1v15::{Signature, VerifyingKey};  
use rsa::pkcs8::DecodePublicKey;  
use rsa::signature::Verifier;  
use rsa::RsaPublicKey;  
use sha1::Sha1;  
use std::collections::HashMap;
```

```

use std::net::IpAddr;
use std::str::FromStr;
use std::sync::RwLock;
use std::time::{SystemTime, UNIX_EPOCH};
use thiserror::Error;
use url::Url;
use wildmatch::WildMatch;

#[derive(Error, Debug)]
pub enum EncodingError {
    #[error("invalid key provided")]
    InvalidKeyError(#[from] rsa::pkcs8::spki::Error),

    #[error("Url parse error")]
    UrlParseError(#[from] url::ParseError),

    #[error("int parse error")]
    IntParseError(#[from] std::num::ParseIntError),

    #[error("Base64 decode error")]
    Base64DecodeError(#[from] base64::DecodeError),

    #[error("Json Parse error")]
    JsonParseError(#[from] json::Error),

    #[error("String from Vec Parse error")]
    StringParseError(#[from] std::string::FromUtf8Error),

    #[error("IP Parse error")]
    IPParseError(#[from] std::net::AddrParseError),

    #[error("IP Network Parse error")]
    IPNetworkParseError(#[from] ipnetwork::IpNetworkError),

    #[error("unknown error")]
    Unknown,
}

struct SignedOptions {
    pub date_less_than: u64, // The expiration date and time for the URL in Unix time
                             // format (in seconds) and Coordinated Universal Time (UTC). Defaults to 1800s from

```

```

now.

    pub date_greater_than: Option<u64>, // An optional start date and time for the
URL in Unix time format (in seconds) and Coordinated Universal Time (UTC).
    pub ip_address: Option<String>, // An optional IP address of the client making
the GET request (can be a range).
}

impl Default for SignedOptions {
    fn default() -> SignedOptions {
        let since_epoch = SystemTime::now().duration_since(UNIX_EPOCH).unwrap();
        SignedOptions {
            date_less_than: since_epoch.as_secs() + 1800,
            date_greater_than: None,
            ip_address: None,
        }
    }
}

fn get_custom_policy(url: &str, options: &SignedOptions) -> String {
    let date_greater_than = options
        .date_greater_than
        .map(|date| format!("{}", "DateGreaterThan":{{"AWS:EpochTime\":"{:?}}}}, date))
        .unwrap_or_default();

    let ip_address = options
        .ip_address
        .clone()
        .map(|ip_address| format!("{}", "IpAddress\":"{{"AWS:SourceIp\":"{:?}}}},
ip_address))
        .unwrap_or_default();

    format!(
        "{{\"Statement\":[{{\"Resource\":"{}\", \"Condition\":"{{\"DateLessThan\":"{{\"AWS:Epo
chTime\":"{:?}}}}}}}}}}",
        url, options.date_less_than, date_greater_than, ip_address
    )
}

fn to_normalize_base64(input: &str) -> String {
    input.replace('-', '+').replace('_', '=').replace('~', '/')
}

```

```

}

fn check_policy(policy_str: &str, request_url: &str, ip: &str) -> Result<bool,
EncodingError> {
    debug!("the policy is \n{}\n", policy_str);
    let since_the_epoch = SystemTime::now()
        .duration_since(UNIX_EPOCH)
        .expect("Time went backwards")
        .as_secs();

    let parsed_policy: json::JsonValue = json::parse(policy_str)?;

    let request_policy_date_less_than =
parsed_policy["Statement"][0]["Condition"]["DateLessThan"]["AWS:EpochTime"].as_u64();
;

    if request_policy_date_less_than.is_some() {
        if since_the_epoch > request_policy_date_less_than.unwrap() {
            debug!("expire in policy is out of date\n");
            return Ok(false);
        }
    } else {
        debug!("expire in policy is missing\n");
        return Ok(false);
    }

    let request_policy_date_greater_than =
parsed_policy["Statement"][0]["Condition"]["DateGreaterThan"]["AWS:EpochTime"].as_u6
4();

    if request_policy_date_greater_than.is_some() {
        if since_the_epoch < request_policy_date_greater_than.unwrap() {
            debug!(
                "the date greater than in policy is less the system datetime {}
{}\n",
                since_the_epoch,
                request_policy_date_greater_than.unwrap()
            );
            return Ok(false);

```



```

    }
}

let request_policy_source_address =
parsed_policy["Statement"][0]["Condition"]["IpAddress"]["AWS:SourceIp"].as_str();

if request_policy_source_address.is_some() {
    // Parse the IP address and subnet
    let ip_address = IpAddr::from_str(ip)?;
    let subnet = IpNetwork::from_str(request_policy_source_address.unwrap())?;
    debug!(
        "ip address in xff:{},in policy resource:{})",
        ip,
        request_policy_source_address.unwrap()
    );
    // Check if the IP address is within the subnet
    if !subnet.contains(ip_address) {
        return Ok(false);
    } else {
        debug!(
            "ip check ok,{} in {}",
            ip,
            request_policy_source_address.unwrap()
        )
    }
}

let request_policy_resouce = parsed_policy["Statement"][0]["Resource"].as_str();

if request_policy_resouce.is_some() {
    if !WildMatch::new(request_policy_resouce.unwrap()).matches(request_url) {
        return Ok(false);
    }
}

return Ok(true);
}

fn verify_cookie(
    url: &str,
    xff_addr: &str,

```

```

    cookie: HashMap<String, String>,
) -> Result<bool, EncodingError> {
    let mut sign_text = String::new();
    if (!cookie.contains_key("CloudFront-Signature"))
        || (!cookie.contains_key("CloudFront-Key-Pair-Id"))
        || ((!cookie.contains_key("CloudFront-Expires"))
            && (!cookie.contains_key("CloudFront-Policy")))
    {
        debug!("missing required cookie");
        return Ok(false);
    }
    debug!("signed cookie check start");

    if !KEYPAIRE
        .read()
        .unwrap()
        .contains_key(cookie.get("CloudFront-Key-Pair-Id").unwrap())
    {
        debug!("the key id is miss in config");
        return Ok(false);
    }

    if cookie.contains_key("CloudFront-Policy") {
        let policy = base64::decode(to_normalize_base64(
            cookie.get("CloudFront-Policy").unwrap(),
        ))?;

        let policy_str = String::from_utf8(policy)?;
        let policy_check_result = check_policy(policy_str.as_str(), url, xff_addr);

        match policy_check_result {
            Ok(v) => {
                if !v {
                    debug!("policy_check_result failed");

                    return Ok(false);
                }
            }
            Err(_m) => {
                debug!("policy check error");
                return Ok(false);
            }
        }
    }
}

```

```

    }

    }

    sign_text = policy_str;
    debug!("Sign Cookie ,the text to be verified is :{}", sign_text);

    let signature =
to_normalize_base64(cookie.get("CloudFront-Signature").unwrap());

    let sig = Base64::decode_vec(signature.as_str());

    match sig {
        Ok(v) => {
            let public_key = RsaPublicKey::from_public_key_pem(
                KEYPAIRE
                    .read()
                    .unwrap()
                    .get(cookie.get("CloudFront-Key-Pair-Id").unwrap())
                    .clone()
                    .unwrap(),
            )?;
            let verifying_key = VerifyingKey::<Shal>::new(public_key);
            let result = verifying_key.verify(
                sign_text.as_bytes(),
                &Signature::try_from(v.as_slice()).unwrap(),
            );
            match result {
                Ok(_v) => {
                    debug!("cookie signature check passed");
                    return Ok(true);
                }
                Err(_m) => {
                    debug!("cookie signature check failed");
                    return Ok(false);
                }
            }
        }
        Err(_m) => {
            debug!("signature decode error");
            return Ok(false);
        }
    }
}

```

```

}

let since_the_epoch = SystemTime::now()
    .duration_since(UNIX_EPOCH)
    .expect("Time went backwards")
    .as_secs();

let request_expire = cookie.get("CloudFront-Expires").unwrap().parse::<u64>()?;
if since_the_epoch > request_expire {
    debug!("{}", "the sign has expired");
    return Ok(false);
}

if sign_text.len() == 0 {
    let options_from_url: SignedOptions = SignedOptions {
        date_less_than: request_expire,
        ..Default::default()
    };

    sign_text = get_custom_policy(url, &options_from_url);
}

debug!("Sign Cookie ,the text to be verified is :{}", sign_text);

let signature = to_normalize_base64(cookie.get("CloudFront-Signature").unwrap());

let sig = Base64::decode_vec(signature.as_str());

match sig {
    Ok(v) => {
        let public_key = RsaPublicKey::from_public_key_pem(
            KEYPAIRE
                .read()
                .unwrap()
                .get(cookie.get("CloudFront-Key-Pair-Id").unwrap())
                .clone()
                .unwrap(),
        )?;
        let verifying_key = VerifyingKey::<Shal>::new(public_key);
        let result = verifying_key.verify(
            sign_text.as_bytes(),
            &Signature::try_from(v.as_slice()).unwrap(),
        );
    }
}

```

```

    );
    match result {
        Ok(_v) => {
            debug!("cookie signature check passed");
            return Ok(true);
        }
        Err(_m) => {
            debug!("cookie signature check failed");
            return Ok(false);
        }
    }
}

Err(_m) => {
    debug!("signature decode error");
    return Ok(false);
}
}
}

fn verify_url(signed_url: &str, xff_addr: &str) -> Result<bool, EncodingError> {
    debug!("the url is: {}", signed_url);
    let mut sign_text = String::new();
    let mut parsed_url = Url::parse(&signed_url)?;
    let hash_query: HashMap<_, _> = parsed_url.query_pairs().into_owned().collect();

    let query: Vec<(String, String)> = parsed_url
        .query_pairs()
        .filter(|(name, _)| name != "Expires")
        .filter(|(name, _)| name != "Policy")
        .filter(|(name, _)| name != "Signature")
        .filter(|(name, _)| name != "Key-Pair-Id")
        .map(|(name, value)| (name.into_owned(), value.into_owned()))
        .collect();

    parsed_url.query_pairs_mut().clear().extend_pairs(&query);

    let size = query.len();
    if size > 0 {
        parsed_url.query_pairs_mut().clear().extend_pairs(query);
    } else {
        parsed_url.set_query(None);
    }
}

```

```

}

let request_policy: Vec<u8>;

let request_url = parsed_url.as_str();
debug!(
    "the resouce url generated from request is:{}\n",
    request_url
);

if ((!hash_query.contains_key("Expires")) &&
(!hash_query.contains_key("Policy")))
    || (!hash_query.contains_key("Signature"))
    || (!hash_query.contains_key("Key-Pair-Id"))
{
    debug!("missing required parameter in query");
    return Ok(false);
}

let request_key_pair_id = hash_query.get("Key-Pair-Id").unwrap().as_str();

if !KEYPAIRE.read().unwrap().contains_key(request_key_pair_id) {
    debug!("the key id is miss in config");
    return Ok(false);
}

let request_signature = hash_query.get("Signature").unwrap().as_str();

// check custom policy

if hash_query.get("Policy").is_some() {
    debug!("using policy sign:{}", hash_query.get("Policy").unwrap());

    request_policy =
base64::decode(to_normalize_base64(hash_query.get("Policy").unwrap()))?;
    sign_text = String::from_utf8(request_policy)?;

    //check policy start

    let policy_check_result = check_policy(sign_text.as_str(), request_url,
xff_addr);

```

```

//check policy end
match policy_check_result {
    Ok(v) => {
        if !v {
            debug!("policy content check failed");
            return Ok(false);
        } else {
            debug!("the text to be verified is : {}", sign_text);
            let signature = to_normalize_base64(request_signature);
            let sig = Base64::decode_vec(signature.as_str());

            match sig {
                Ok(v) => {
                    let public_key = RsaPublicKey::from_public_key_pem(
                        KEYPAIRE
                            .read()
                            .unwrap()
                            .get(request_key_pair_id)
                            .clone()
                            .unwrap(),
                    )?;
                    let verifying_key =
VerifyingKey::<Sha1>::new(public_key);
                    let result = verifying_key.verify(
                        sign_text.as_bytes(),
                        &Signature::try_from(v.as_slice()).unwrap(),
                    );
                    match result {
                        Ok(_v) => return Ok(true),
                        Err(_m) => return Ok(false),
                    }
                }
                Err(_m) => return Ok(false),
            }
        }
    }
    Err(_m) => {
        debug!("policy decode error");
        return Ok(false);
    }
}

```

```

}

//check canned policy

let request_expire = hash_query.get("Expires").unwrap().parse::()?;
let since_the_epoch = SystemTime::now()
    .duration_since(UNIX_EPOCH)
    .expect("Time went backwards")
    .as_secs();
if since_the_epoch > request_expire {
    debug!("{}", "the sign has expired");
    return Ok(false);
}

if sign_text.len() == 0 {
    let options_from_url: SignedOptions = SignedOptions {
        date_less_than: request_expire,
        ..Default::default()
    };

    sign_text = get_custom_policy(request_url, &options_from_url);
}

debug!("{}", "the text to be verified is : {}", sign_text);
let signature = to_normalize_base64(request_signature);
let sig = Base64::decode_vec(signature.as_str());

match sig {
    Ok(v) => {
        let public_key = RsaPublicKey::from_public_key_pem(
            KEYPAIR
                .read()
                .unwrap()
                .get(request_key_pair_id)
                .clone()
                .unwrap(),
        )?;
        let verifying_key = VerifyingKey::Sh1::new(public_key);
        let result = verifying_key.verify(
            sign_text.as_bytes(),
            &Signature::try_from(v.as_slice()).unwrap(),
        );
        match result {

```



```

        Ok(_v) => return Ok(true),
        Err(_m) => return Ok(false),
    }
}

Err(_m) => return Ok(false),
}
}

struct DemoPluginRoot {}

//static config: &'static HashMap<String, String> =
HashMap<String,String>::with_capacity(32);

impl<'b> RootContext for DemoPluginRoot {
    fn on_configure(&mut self, _: usize) -> bool {
        if let Some(config_bytes) = self.get_plugin_configuration() {
            let config_str = String::from_utf8(config_bytes).unwrap();
            let configs: Vec<&str> = config_str.split(";").collect();
            for config_itme in configs {
                let parts: Vec<&str> = config_itme.splitn(2, "=").collect();
                if parts.len() == 2 {
                    let name = parts[0].trim().to_string();
                    let value = parts[1].trim().to_string();
                    KEYPAIRE.write().unwrap().insert(name, value);
                }
            }
        }
        true
    }

    fn create_http_context(&self, _: u32) -> Option<Box<dyn HttpContext>> {
        Some(Box::new(DemoPlugin {}))

        //Some(Box::new(DemoPlugin::from_root(&self)))
    }

    fn get_type(&self) -> Option<ContextType> {
        Some(ContextType::HttpContext)
    }
}

impl Context for DemoPluginRoot {}

```

```

impl Context for DemoPlugin {}

struct DemoPlugin {}

/*
impl<'a> DemoPlugin<'a> {
    fn from_root(root: &'a DemoPluginRoot) -> DemoPlugin<'a> {
        let plugin = DemoPlugin {
            config: &root.config,
        };
        return plugin;
    }
}
*/

impl HttpContext for DemoPlugin {
    fn on_http_request_headers(&mut self, _: usize, _: bool) -> Action {
        //let http_host = self.config.get("host").unwrap().clone();
        let http_host = self.get_http_request_header(":authority").unwrap();
        let http_scheme = self.get_http_request_header(":scheme").unwrap();
        let http_path = self.get_http_request_header(":path").unwrap();
        let http_xff = self.get_http_request_header("x-forwarded-for").unwrap();
        let addr = http_xff.as_str();
        debug!(
            "the request is {}://{{{}}}, from ip address {}",
            http_scheme, http_host, http_path, addr
        );

        //self.config.remove("host");

        // sign url validation
        let signed_url = format!("{}://{{{}}}", http_scheme, http_host, http_path);

        debug!("checking sign url");
        let result = verify_url(signed_url.as_str(), addr);

        match result {
            Ok(v) => {
                if v {
                    debug!("signed url check is ok");
                    return Action::Continue;
                } else {

```

```

        debug!("signed url check failed due to signature not match");
    }
}
Err(msg) => {
    debug!("sign url check failed on:{}", msg);
}
}
debug!("checking cookie");

let mut cookie_map: HashMap<String, String> = HashMap::new();
if let Some(cookie_header) = self.get_http_request_header("Cookie") {
    let cookies: Vec<&str> = cookie_header.split(";").collect();
    for cookie in cookies {
        let parts: Vec<&str> = cookie.splitn(2, "=").collect();
        if parts.len() == 2 {
            let name = parts[0].trim().to_string();
            let value = parts[1].trim().to_string();
            debug!("find cookie:{{}}={{}}", name, value);
            cookie_map.insert(name, value);
        }
    }
}
let result = verify_cookie(signed_url.as_str(), addr, cookie_map);

match result {
    Ok(v) => {
        if v {
            debug!("signed cookie check pass");
            return Action::Continue;
        } else {
            self.send_http_response(403, vec![], Some(b"Access
forbidden.\n"));

            return Action::Pause;
        }
    }
}
Err(msg) => {
    debug!("verify cookie failed on:{}", msg);
    self.send_http_response(403, vec![], Some(b"Access
forbidden.\n"));

    return Action::Pause;
}
}

```

```

    }

    self.send_http_response(403, vec![], Some(b"Access forbidden.\n"));
    return Action::Pause;
}
}

lazy_static! {
    static ref KEYP Aire: RwLock<HashMap<String, String>> =
RwLock::new(HashMap::new());
}

proxy_wasm::main! {{
    proxy_wasm::set_log_level(LogLevel::Trace);
    proxy_wasm::set_root_context(|_| -> Box<dyn RootContext> {
Box::new(DemoPluginRoot{
    })
    });
}}

```

Rc version

```

use log::debug;
use proxy_wasm::traits::*;
use proxy_wasm::types::*;

use base64ct::{Base64, Encoding};
use ipnetwork::IpNetwork;
use json;
use rsa::pkcs1v15::{Signature, VerifyingKey};
use rsa::pkcs8::DecodePublicKey;
use rsa::signature::Verifier;
use rsa::RsaPublicKey;
use sha1::Sha1;
use std::collections::HashMap;

```

```

use std::net::IpAddr;
use std::rc::Rc;
use std::str::FromStr;
use std::time::{SystemTime, UNIX_EPOCH};
use thiserror::Error;
use url::Url;
use wildmatch::WildMatch;

#[derive(Error, Debug)]
pub enum EncodingError {
    #[error("invalid key provided")]
    InvalidKeyError(#[from] rsa::pkcs8::spki::Error),

    #[error("Url parse error")]
    UrlParseError(#[from] url::ParseError),

    #[error("int parse error")]
    IntParseError(#[from] std::num::ParseIntError),

    #[error("Base64 decode error")]
    Base64DecodeError(#[from] base64::DecodeError),

    #[error("Json Parse error")]
    JsonParseError(#[from] json::Error),

    #[error("String from Vec Parse error")]
    StringParseError(#[from] std::string::FromUtf8Error),

    #[error("IP Parse error")]
    IPParseError(#[from] std::net::AddrParseError),

    #[error("IP Network Parse error")]
    IPNetworkParseError(#[from] ipnetwork::IpNetworkError),

    #[error("unknown error")]
    Unknown,
}

struct SignedOptions {
    pub date_less_than: u64, // The expiration date and time for the URL in Unix time
                             // format (in seconds) and Coordinated Universal Time (UTC). Defaults to 1800s from

```

```

now.

    pub date_greater_than: Option<u64>, // An optional start date and time for the
URL in Unix time format (in seconds) and Coordinated Universal Time (UTC).
    pub ip_address: Option<String>, // An optional IP address of the client making
the GET request (can be a range).
}

impl Default for SignedOptions {
    fn default() -> SignedOptions {
        let since_epoch = SystemTime::now().duration_since(UNIX_EPOCH).unwrap();
        SignedOptions {
            date_less_than: since_epoch.as_secs() + 1800,
            date_greater_than: None,
            ip_address: None,
        }
    }
}

fn get_custom_policy(url: &str, options: &SignedOptions) -> String {
    let date_greater_than = options
        .date_greater_than
        .map(|date| format!("{}", "DateGreaterThan":{{"AWS:EpochTime\":"{:?}}}", date))
        .unwrap_or_default();

    let ip_address = options
        .ip_address
        .clone()
        .map(|ip_address| format!("{}", "IpAddress\":"{{"AWS:SourceIp\":"{:?}}}",
ip_address))
        .unwrap_or_default();

    format!(
        "{{\"Statement\":[{{\"Resource\":"{}\", \"Condition\":"{{\"DateLessThan\":"{{\"AWS:Epo
chTime\":"{:?}}}}}}]]}}",
        url, options.date_less_than, date_greater_than, ip_address
    )
}

fn to_normalize_base64(input: &str) -> String {
    input.replace('-', '+').replace('_', '=').replace('~', '/')
}

```

```

}

proxy_wasm::main! {{
proxy_wasm::set_log_level(LogLevel::Trace);
proxy_wasm::set_root_context(|_| -> Box<dyn RootContext> {
Box::new(DemoPluginRoot {
config: Rc::new(HashMap::new()),
})
});
}}

fn check_policy(policy_str: &str, request_url: &str, ip: &str) -> Result<bool,
EncodingError> {
    debug!("the policy is \n{}\n", policy_str);
    let since_the_epoch = SystemTime::now()
        .duration_since(UNIX_EPOCH)
        .expect("Time went backwards")
        .as_secs();

    let parsed_policy: json::JsonValue = json::parse(policy_str)?;

    let request_policy_date_less_than =

parsed_policy["Statement"][0]["Condition"]["DateLessThan"]["AWS:EpochTime"].as_u64()
;

    if request_policy_date_less_than.is_some() {
        if since_the_epoch > request_policy_date_less_than.unwrap() {
            debug!("expire in policy is out of date\n");
            return Ok(false);
        }
    } else {
        debug!("expire in policy is missing\n");
        return Ok(false);
    }

    let request_policy_date_greater_than =

parsed_policy["Statement"][0]["Condition"]["DateGreaterThan"]["AWS:EpochTime"].as_u6
4();

    if request_policy_date_greater_than.is_some() {

```

```

        if since_the_epoch < request_policy_date_greater_than.unwrap() {
            debug!(
                "the date greater than in policy is less the system datetime {}",
                since_the_epoch,
                request_policy_date_greater_than.unwrap()
            );
            return Ok(false);
        }
    }

    let request_policy_source_address =
parsed_policy["Statement"][0]["Condition"]["IpAddress"]["AWS:SourceIp"].as_str();

    if request_policy_source_address.is_some() {
        // Parse the IP address and subnet
        let ip_address = IpAddr::from_str(ip)?;
        let subnet = IpNetwork::from_str(request_policy_source_address.unwrap())?;
        debug!(
            "ip address in xff:{}, in policy resource:{}",
            ip,
            request_policy_source_address.unwrap()
        );
        // Check if the IP address is within the subnet
        if !subnet.contains(ip_address) {
            return Ok(false);
        } else {
            debug!(
                "ip check ok, {} in {}",
                ip,
                request_policy_source_address.unwrap()
            )
        }
    }

    let request_policy_resouce = parsed_policy["Statement"][0]["Resource"].as_str();

    if request_policy_resouce.is_some() {
        if !WildMatch::new(request_policy_resouce.unwrap()).matches(request_url) {
            return Ok(false);
        }
    }

```



```

    }

    return Ok(true);
}

fn verify_cookie(
    url: &str,
    public_key: &HashMap<String, String>,
    xff_addr: &str,
    cookie: HashMap<String, String>,
) -> Result<bool, EncodingError> {
    let mut sign_text = String::new();
    if (!cookie.contains_key("CloudFront-Signature"))
        || (!cookie.contains_key("CloudFront-Key-Pair-Id"))
        || ((!cookie.contains_key("CloudFront-Expires"))
            && (!cookie.contains_key("CloudFront-Policy"))))
    {
        debug!("missing required cookie");
        return Ok(false);
    }
    debug!("signed cookie check start");

    if !public_key.contains_key(cookie.get("CloudFront-Key-Pair-Id").unwrap()) {
        debug!("the key id is miss in config");
        return Ok(false);
    }

    if cookie.contains_key("CloudFront-Policy") {
        let policy = base64::decode(to_normalize_base64(
            cookie.get("CloudFront-Policy").unwrap(),
        ))?;

        let policy_str = String::from_utf8(policy)?;
        let policy_check_result = check_policy(policy_str.as_str(), url, xff_addr);

        match policy_check_result {
            Ok(v) => {
                if !v {
                    debug!("policy_check_result failed");

                    return Ok(false);
                }
            }
        }
    }
}

```

```

    }
}
Err(_m) => {
    debug!("policy check error");
    return Ok(false);
}
}

sign_text = policy_str;
debug!("Sign Cookie ,the text to be verified is :{}", sign_text);

let signature =
to_normalize_base64(cookie.get("CloudFront-Signature").unwrap());

let sig = Base64::decode_vec(signature.as_str());

match sig {
    Ok(v) => {
        let public_key = RsaPublicKey::from_public_key_pem(
            public_key
                .get(cookie.get("CloudFront-Key-Pair-Id").unwrap())
                .clone()
                .unwrap(),
        )?;
        let verifying_key = VerifyingKey::<Shal>::new(public_key);
        let result = verifying_key.verify(
            sign_text.as_bytes(),
            &Signature::try_from(v.as_slice()).unwrap(),
        );
        match result {
            Ok(_v) => {
                debug!("cookie signature check passed");
                return Ok(true);
            }
            Err(_m) => {
                debug!("cookie signature check failed");
                return Ok(false);
            }
        }
    }
}
Err(_m) => {
    debug!("signature decode error");
}

```

```

        return Ok(false);
    }
}

let since_the_epoch = SystemTime::now()
    .duration_since(UNIX_EPOCH)
    .expect("Time went backwards")
    .as_secs();

let request_expire = cookie.get("CloudFront-Expires").unwrap().parse::<u64>()?;
if since_the_epoch > request_expire {
    debug!("{}", "the sign has expired");
    return Ok(false);
}

if sign_text.len() == 0 {
    let options_from_url: SignedOptions = SignedOptions {
        date_less_than: request_expire,
        ..Default::default()
    };

    sign_text = get_custom_policy(url, &options_from_url);
}

debug!("Sign Cookie ,the text to be verified is :{}", sign_text);

let signature = to_normalize_base64(cookie.get("CloudFront-Signature").unwrap());

let sig = Base64::decode_vec(signature.as_str());

match sig {
    Ok(v) => {
        let public_key = RsaPublicKey::from_public_key_pem(
            public_key
                .get(cookie.get("CloudFront-Key-Pair-Id").unwrap())
                .clone()
                .unwrap(),
        )?;
        let verifying_key = VerifyingKey::<Sha1>::new(public_key);
        let result = verifying_key.verify(
            sign_text.as_bytes(),

```

```

        &Signature::try_from(v.as_slice()).unwrap(),
    );
    match result {
        Ok(_v) => {
            debug!("cookie signature check passed");
            return Ok(true);
        }
        Err(_m) => {
            debug!("cookie signature check failed");
            return Ok(false);
        }
    }
}

Err(_m) => {
    debug!("signature decode error");
    return Ok(false);
}
}

fn verify_url(
    signed_url: &str,
    public_key: &HashMap<String, String>,
    xff_addr: &str,
) -> Result<bool, EncodingError> {
    debug!("the url is: {}", signed_url);
    let mut sign_text = String::new();
    let mut parsed_url = Url::parse(&signed_url)?;
    let hash_query: HashMap<_, _> = parsed_url.query_pairs().into_owned().collect();

    let query: Vec<(String, String)> = parsed_url
        .query_pairs()
        .filter(|(name, _)| name != "Expires")
        .filter(|(name, _)| name != "Policy")
        .filter(|(name, _)| name != "Signature")
        .filter(|(name, _)| name != "Key-Pair-Id")
        .map(|(name, value)| (name.into_owned(), value.into_owned()))
        .collect();

    parsed_url.query_pairs_mut().clear().extend_pairs(&query);

```

```

let size = query.len();
if size > 0 {
    parsed_url.query_pairs_mut().clear().extend_pairs(query);
} else {
    parsed_url.set_query(None);
}

let request_policy: Vec<u8>;

let request_url = parsed_url.as_str();
debug!(
    "the resouce url generated from request is:{}\n",
    request_url
);

if ((!hash_query.contains_key("Expires")) &&
(!hash_query.contains_key("Policy")))
    || (!hash_query.contains_key("Signature"))
    || (!hash_query.contains_key("Key-Pair-Id"))
{
    debug!("missing required parameter in query");
    return Ok(false);
}

let request_key_pair_id = hash_query.get("Key-Pair-Id").unwrap().as_str();

if !public_key.contains_key(request_key_pair_id) {
    debug!("the key id is miss in config");
    return Ok(false);
}

let request_signature = hash_query.get("Signature").unwrap().as_str();

// check custom policy

if hash_query.get("Policy").is_some() {
    debug!("using policy sign:{}", hash_query.get("Policy").unwrap());

    request_policy =
base64::decode(to_normalize_base64(hash_query.get("Policy").unwrap()))?;
    sign_text = String::from_utf8(request_policy)?;

```

```

//check policy start

let policy_check_result = check_policy(sign_text.as_str(), request_url,
xff_addr);

//check policy end
match policy_check_result {
    Ok(v) => {
        if !v {
            debug!("policy content check failed");
            return Ok(false);
        } else {
            debug!("the text to be verified is : {}", sign_text);
            let signature = to_normalize_base64(request_signature);
            let sig = Base64::decode_vec(signature.as_str());

            match sig {
                Ok(v) => {
                    let public_key = RsaPublicKey::from_public_key_pem(
                        public_key.get(request_key_pair_id).clone().unwrap(),
                    )?;
                    let verifying_key =
VerifyingKey::<Sha1>::new(public_key);
                    let result = verifying_key.verify(
                        sign_text.as_bytes(),
                        &Signature::try_from(v.as_slice()).unwrap(),
                    );
                    match result {
                        Ok(_v) => return Ok(true),
                        Err(_m) => return Ok(false),
                    }
                }
                Err(_m) => return Ok(false),
            }
        }
    }
    Err(_m) => {
        debug!("policy decode error");
        return Ok(false);
    }
}

```

```

}

//check canned policy

let request_expire = hash_query.get("Expires").unwrap().parse::()?;
let since_the_epoch = SystemTime::now()
    .duration_since(UNIX_EPOCH)
    .expect("Time went backwards")
    .as_secs();
if since_the_epoch > request_expire {
    debug!("{}", "the sign has expired");
    return Ok(false);
}

if sign_text.len() == 0 {
    let options_from_url: SignedOptions = SignedOptions {
        date_less_than: request_expire,
        ..Default::default()
    };

    sign_text = get_custom_policy(request_url, &options_from_url);
}

debug!("{}", "the text to be verified is : {}", sign_text);
let signature = to_normalize_base64(request_signature);
let sig = Base64::decode_vec(signature.as_str());

match sig {
    Ok(v) => {
        let public_key = RsaPublicKey::from_public_key_pem(
            public_key.get(request_key_pair_id).clone().unwrap(),
        )?;
        let verifying_key = VerifyingKey::<Sha1>::new(public_key);
        let result = verifying_key.verify(
            sign_text.as_bytes(),
            &Signature::try_from(v.as_slice()).unwrap(),
        );
        match result {
            Ok(_v) => return Ok(true),
            Err(_m) => return Ok(false),
        }
    }
    Err(_m) => return Ok(false),
}

```

```

    }
}

struct DemoPluginRoot {
    config: Rc<HashMap<String, String>>,
}

impl RootContext for DemoPluginRoot {
    fn on_configure(&mut self, _: usize) -> bool {
        if let Some(config_bytes) = self.get_plugin_configuration() {
            let config_str = String::from_utf8(config_bytes).unwrap();
            let configs: Vec<&str> = config_str.split(";").collect();
            let mut keys: HashMap<String, String> = HashMap::new();
            for config_itme in configs {
                let parts: Vec<&str> = config_itme.splitn(2, "=").collect();
                if parts.len() == 2 {
                    let name = parts[0].trim().to_string();
                    let value = parts[1].trim().to_string();
                    keys.insert(name, value);
                }
            }
            self.config = Rc::new(keys)
        }
        true
    }

    fn create_http_context(&self, _: u32) -> Option<Box<dyn HttpContext>> {
        Some(Box::new(DemoPlugin {
            config: self.config.clone(),
        }))
    }

    fn get_type(&self) -> Option<ContextType> {
        Some(ContextType::HttpContext)
    }
}

impl Context for DemoPluginRoot {}

struct DemoPlugin {
    config: Rc<HashMap<String, String>>,
}

```



```

impl HttpContext for DemoPlugin {
    fn on_http_request_headers(&mut self, _: usize, _: bool) -> Action {
        let http_host = self.get_http_request_header(":authority").unwrap();
        let http_scheme = self.get_http_request_header(":scheme").unwrap();
        let http_path = self.get_http_request_header(":path").unwrap();
        let http_xff = self.get_http_request_header("x-forwarded-for").unwrap();
        let addr = http_xff.as_str();
        debug!(
            "the request is {}://{{{}}}, from ip address {}",
            http_scheme, http_host, http_path, addr
        );

        let signed_url = format!("{}://{{{}}}", http_scheme, http_path);

        debug!("checking sign url");
        let result = verify_url(signed_url.as_str(), &self.config, addr);

        match result {
            Ok(v) => {
                if v {
                    debug!("signed url check is ok");
                    return Action::Continue;
                } else {
                    debug!("signed url check failed due to signature not match");
                }
            }
            Err(msg) => {
                debug!("sign url check failed on:{}", msg);
            }
        }
        debug!("checking cookie");

        let mut cookie_map: HashMap<String, String> = HashMap::new();
        if let Some(cookie_header) = self.get_http_request_header("Cookie") {
            let cookies: Vec<&str> = cookie_header.split(";").collect();
            for cookie in cookies {
                let parts: Vec<&str> = cookie.splitn(2, "=").collect();
                if parts.len() == 2 {
                    let name = parts[0].trim().to_string();
                    let value = parts[1].trim().to_string();

```

```

        debug!("find cookie:{}={}", name, value);
        cookie_map.insert(name, value);
    }
}

let result = verify_cookie(signed_url.as_str(), &self.config, addr,
cookie_map);

match result {
    Ok(v) => {
        if v {
            debug!("signed cookie check pass");
            return Action::Continue;
        } else {
            self.send_http_response(403, vec![], Some(b"Access
forbidden.\n"));

            return Action::Pause;
        }
    }
    Err(msg) => {
        debug!("verify cookie failed on:{}", msg);
        self.send_http_response(403, vec![], Some(b"Access
forbidden.\n"));

        return Action::Pause;
    }
}

self.send_http_response(403, vec![], Some(b"Access forbidden.\n"));
return Action::Pause;
}
}

impl Context for DemoPlugin {}

```